

1st year PhD SOTA progress Report for PhD Thesis: Weight space regression for efficient continual learning

Aymen Tlili^{a,1}

^aData engineering and semantics Lab: Faculty of Sciences of Sfax / STIC:ENET'COM

Supervising Professor: Bassem Ben Hamed

Abstract—In this report, I elaborate the methodology used to train our Transformer Architecture for Task Merging via weight regression or sequence-to-sequence modeling, as well as observations and specificities of using this task in the context of continual learning with the ultimate aim being uncovering what constitutes learning in a neural net and what's benign noise.

Keywords— $\LaTeX$ Computer Vision, Hyper-representation, AutoML, MNIST, Continual learning, Sequence modeling

1. Introduction

Eminating from needs based on continuous learning and fast DevOps operations that require adapting models to an ever-flowing amount of data: The task of keeping a model up-to-date with current patterns and trends without losing previous knowledge and generalization power is a major challenge as the hardware cost, time investment, expertise needed and the environmental repercussions of training good-fit models that are able to perform both in and out of the learning dataset distribution.

Based on existing hypotheses that neural networks are just a compact format of the representations possible for a dataset and admissible conjecture about the universal approximation theorem benefits from repeating certain challenging examples that lie on the decision boundary more than easy-to-fit ones, We opt to focus on a subclass of deep neural networks known as convolutional NNs. These models are moderately adept at resolving tasks like classification/regression or unsupervised context ones and with such diversity we fix a singular architecture and vary the weights and biases that are supposed carriers of information about the geometry of the learned manifold.

Previous Hyper-representation works focused on denoising parameters and strategically sampling a hidden representation of these CNNs from a meta-learner Transformer and then finetuning them into functional learners but used a set of CNNs that are denoted 'model zoo' which were only exposed to data superficially hence underfitting caused bad Accuracy metric results on classification tasks. The latent representations in the hidden neural network are usually the most spectrally compact and filled with informative features critical to the output's quality. In the transformer AutoEncoder used this was the (bottle-)neck represented as a dense Layer relating a downsampling encoder fed poor quality CNNs and its mirroring upsampling decoder which produced the series of positionally tokenized denoised CNN parameters.

This model zoo is ill-fit for our task of Merging knowledge between 2 CNNs to produce a 3rd CNN with same number of parameters as that zoo's models are no better than coin flippers whereas Continual-learning assumes the input models have a task-incremental or class-incremental setting or even same tasks but with concept drifting features i.e. time-varying distribution making old models useless against new tasks. This is quickly remedied by finetuning with fresh/new data but the compromise with that is sacrificing knowledge from earlier model version and expensive-to-store older models and giving rise to the need to resort to one or hybrids of 3 categories of strategies to combat catastrophic forgetting and maintain old representations in new models:

- Regularization methods: L1 constrain new models' parameters to not stray too far away from their counterparts in older models.
- Replay methods: save Information-rich/Class-representative examples or even GANs to produce old data on demand
- Architecture-incremental methods: Keep an adapter/task-specific layer stored

2. Contributions:

- Show the effect of the loss function choice on the latent representations performance just after Inference and after finetuning [continuation of Layer-wise-Loss normalization proposed in hyperrepresentation paper]
- Propose a new CNNs Zoo best-fit for Class-incremental learning with proficient learners with high accuracy ceilings [unlike hyper-representation paper, have good accuracy and trained for 40 epochs with early stopping vs 5-10-25 epoch weak learning rate models]
 - Merging knowledge
 - Subtracting knowledge(model unlearning)
 - minimize the reliance on real data (Federated Learning application)
- Showcase the **topological/spectral differences** in the presence or absence of certain classes/class combinations and derive optimizations methods from them (currently using loss terms)
- Clearly Separate the topological(PH/in-layer distribution) and spectral(eigenvalue/FFT distribution) that allow finetuning to reach in 200-500 steps of the first pair of epochs high performance between Transformer predicted weights and finetuned/ground-truth models.
- Find What properties are maintained between the ground truth and finetuned models that we consider intrinsic for performance
- See what eventual combination or singular Loss best expresses what the meta learner should select in a sequence of weights
- Eventually sparsely feed only necessary weights to achieve a certain task and maintain generalisability across out-of-distribution classes
- Eventually use KANs to explicitly derive the formula between predicted and finetuned models

Good Performance is rated across multiple facets for what constitutes Learning

- CNN accuracy percentage after prediction IID and OOD
- CNN accuracy percentage after finetuning IID and OOD
- How fast the predicted CNN catches up to its **parents'** accuracy
- How stable the predicted CNNs Accuracy is after each epoch of training (no deterioration as it continues to train)
- Transformer Loss value and how well it's correlated to the CNN performance
- How Clearly separable are Classes [do we need to introduce something like diffusion-aware bernoulli mask to avoid the model giving the same output regardless of input? if this prediction's collapse to a singular model is benign, how good is that model?]
- Continual learning metrics like forgetting(loss of acc on old) and forward-learning(how older classes [presence/Absence or order of appearance] contributed to making future classes easier to learn)

3. Dataset of weights (name pending):

3.1. Setting:

We define the images I_C and their respective labels Y_C with $Y = [Y_0, Y_1, \dots, Y_9]$ corresponding to the specific Classes C_i from the MNIST Dataset. The following sub-section elaborates how we fit a convolutional neural network model $CNN^{[Y_C]}$ hence having weights $w_{[g,i,e]}^{[l,Y_C]}$ with l being the index of the Layer in our 3-convolution Layers 2-dense Layers fixed architecture model. We limit this work temporarily to working with kaiming uniform initialization zoo as it had a good median of results and least outliers during training

Possible values	0	1	2	3	4	5
activation A	gelu	relu	silu	leakyrelu	sigmoid	tanh
checkpoint Epoch e	11	16	21	26	31	36
initialization i	xavier uniform	xavier normal	uniform	normal	kaiming normal	kaiming uniform

label	0	1	2	3	4	5	6	7	8	9	gelu	relu	silu	tanh	sigmoid	leakyrelu	weight 0	weight 1	weight 2	bias 2462	bias 2463	Accuracy	epoch
1	[0.1]	1	0	0	0	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	0.094684735	-0.7905626	-0.4464047	-0.13655047	-0.32772434	98.64333333333333	36
2	[0.2]	1	0	1	0	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.0496985	-0.061642155	-0.0929891	0.19706774	-0.36267012	98.90666666666668	36
3	[0.3]	1	0	0	1	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.48978	-0.005564362	-0.105482616	0.088529125	-0.2061299	98.37666666666668	36
4	[0.4]	1	0	0	0	1	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	0.29166335	0.5139673	-0.07567718	-0.20371349	-0.28380706	98.91166666666668	36
5	[0.5]	1	0	0	0	0	1	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.24452844	0.029974932	0.52706844	0.08827021	-0.0042240657	98.86833333333334	36
6	[0.6]	1	0	0	0	0	0	1	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.0375004	-0.4844076	-0.27842575	-0.085808694	0.11616701	98.755	36
7	[0.7]	1	0	0	0	0	0	0	1	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.052341357	-0.064843126	0.19656326	-0.237271	-0.04033807	98.84666666666668	36
8	[0.8]	1	0	0	0	0	0	0	0	1	1.0	0.0	0.0	0.0	0.0	0.0	-0.53858924	-0.3099806	-0.19575508	-0.45016676	-0.29142836	98.79	36
9	[0.9]	1	0	0	0	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	0.44759452	0.30329853	0.07768698	-0.14843176	-0.29648837	98.67333333333332	36
10	[1.2]	0	1	1	0	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.059722804	0.360608	0.35631633	-0.07278538	-0.06793964	98.685	36
11	[1.3]	0	1	0	1	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.04179168	-0.21159662	-0.52923477	-0.5364756	-0.000998338	98.55	36
12	[1.4]	0	1	0	0	1	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	0.1668191	0.41012523	-0.24427213	-0.022639548	0.062193777	97.725	36
13	[1.5]	0	1	0	0	0	1	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	0.014314616	-0.08713338	0.21204941	-0.47954503	-0.218975	98.53666666666668	36
14	[1.6]	0	1	0	0	0	0	1	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-1.000273	-0.22925067	-0.32118472	-0.24637736	-0.040949624	98.51166666666668	36
15	[1.7]	0	1	0	0	0	0	0	1	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.21473292	0.23508567	-0.26331523	-0.6218089	-0.33946937	98.85166666666667	36
16	[1.8]	0	1	0	0	0	0	0	0	1	1.0	0.0	0.0	0.0	0.0	0.0	0.23771928	-0.1500905	0.27912033	-0.35365957	-0.37145048	98.52	36
17	[1.9]	0	1	0	0	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.21042645	-0.08642456	0.22186308	0.2798944	-0.19171506	98.90833333333332	36
18	[2.3]	0	0	1	1	0	0	0	0	0	1.0	0.0	0.0	0.0	0.0	0.0	-0.09061946	0.1488133	0.47424597	-0.45934552	-0.5223393	98.695	36

(a) 36468 row/model Kaiming Uniform init zoo early columns and Experience identifier label 1-hot encoding (b) Last columns and accuracy and the epoch identifier

Figure 1. The .csv format opted for in our zoo

3.2. Training:

We train every subcombination possible of CNNs classification on 10-label MNIST Dataset to clearly target the effect of the presence of a class or it's absence i.e models operating on 2-10 classes. We vary the initialization distribution and activation used in the weights for diversity and robustness of the learned models. That being said, we fix the initial seed, making models sharing the same init have the same exact sampling first origin checkpoint which will vary eventually on classes it's exposed to.

<pre> class CNN(nn.Module): def __init__(self): self.channels_in = 1 self.n_layers = 10 self.init_type = "uniform" self.dropout = 0.5 self.__init__() self.module_list = nn.ModuleList() self.module_list.append(nn.Conv2d(1, 8, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Conv2d(8, 6, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Conv2d(6, 4, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Dropout(self.dropout)) self.module_list.append(nn.Conv2d(4, 3, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Conv2d(3, 2, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Conv2d(2, 1, 5, 1)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Flatten()) self.module_list.append(nn.Linear(1 * 5 * 5 * 1, 20)) self.module_list.append(nn.ReLU(inplace=True)) self.module_list.append(nn.Dropout(self.dropout)) self.module_list.append(nn.Linear(20, 10)) self.initialize_weights(self.init_type) </pre>	Weights weight (8×1×5×5) bias (8) Weights weight (6×8×5×5) bias (6) Weights weight (4×6×2×2) bias (4) Weights weight (20×36) bias (20) Weights weight (10×20) bias (10)
---	--

(a) Class CNN declaration (b) Layer declaration Weights and channels

Figure 2. CNN architecture

The file 'Silu.py' shows an example of a script we let run on the same remote server. On other machines we change the activation in the activation list. The training is done until convergence and satisfaction conditions are met.

- 40 epochs of training are originally declared
- checkpointing happens only every 5 epochs after the 10th and only if Validation-set accuracy for that **Experience** (set of classes chosen).
- Early stopping rules :A stagnation counter increments until it meets a patience parameters. If model validation accuracy 'stagnates' for 3 epochs. stagnation is defined as staying within a range of a Margin 0.05% accuracy from the last epoch's accuracy.
- The early stopping means we can have CNNs scattered over different bins of epochs. The exact epoch is saved along the checkpoint but for the formatting of the 'Merged Zoo.csv' we save the model weights along it's closest predecessor bin representer. *For example* : a model exiting on epoch 19 is saved along models in the epoch 15 bin and is not trained or logged further than 19 epochs

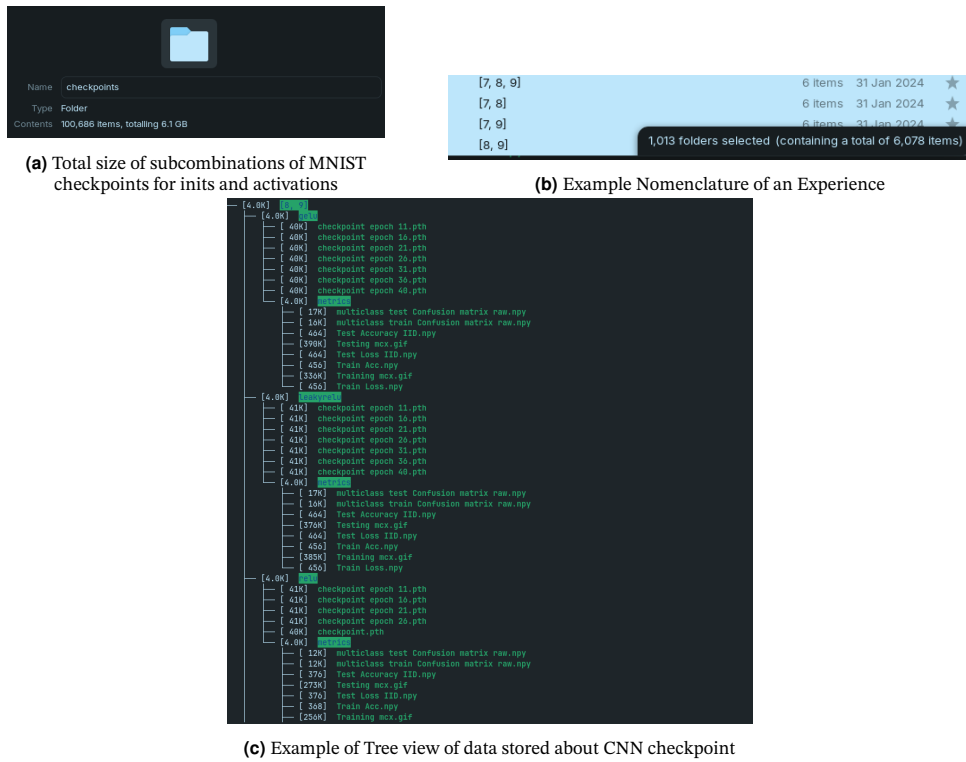


Figure 3. Showcasing the Dataset checkpoints before building .csv format .

The architecture is the 1-channel architecture used in the hyper-representation paper. We log the accuracy and loss of every single CNN in numpy arrays along of the multiclass confusion matrix and create a gif out of them.

[details about optimizer Adam+scheduler CyclicLR choice/learning rate/usage of avalanche to create SplitMnist and save each class' images in a separate folder can be explained later but as per standard cross-entropy was the loss and no data augmentation was used]

3.3. Declaring Scenarios:

As explained above , we have early stopping for the CNNs meaning that some don't make it to the final 40 epochs ,hence a sub-scenario is defined with

- Activation **A**
- Set of Experiences **E**
- a parameter of the number of allowed overlapping classes **m**

The 'Create training scenarios.ipynb' elaborates how we created 2 sets of scenarios

1. sub-scenario type 1: from [0-9] with 0,1,2 overlapping classes
2. sub-scenario type 2: from [0-5] with up to 4 overlapping classes

A training Scenario is finally decided by taking $S=(S_b, \text{epoch})$ with all initializations included and that is used to declare the run title in WeightsAndBiases.com (Wandb) our logging tool and using our CustomDataset Class.

The idea to have overlapping classes was to see if we can identify a topological or spectral denominator or to see if it would make learning Weights that DID NOT require finetuning easier. For now all experiments are done with 0 overlap and so the fusion of knowledge happens across pairs and by varying the the pairs themselves we force the Transformer to learn to Fuse independently of the weight input.

3.4. Statistical Analysis:

every layer has it's own mean and std of values

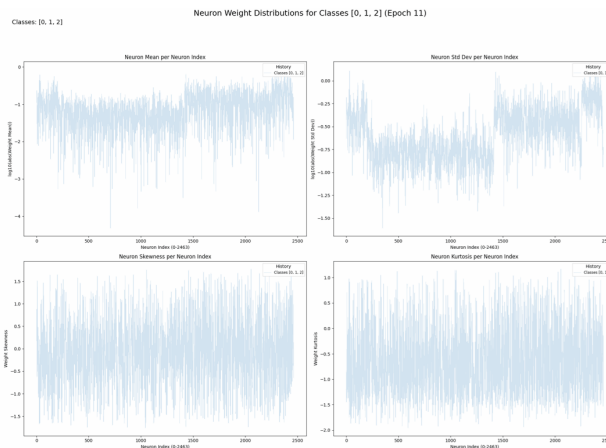


Figure 4. Statistics on models [0,1,2]

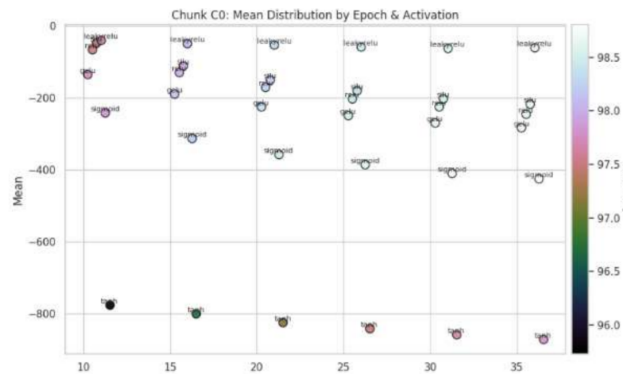
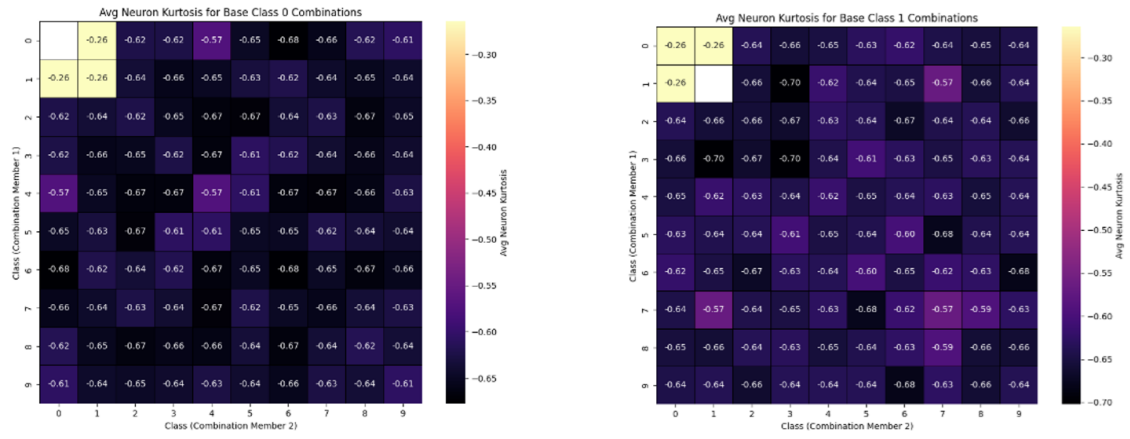


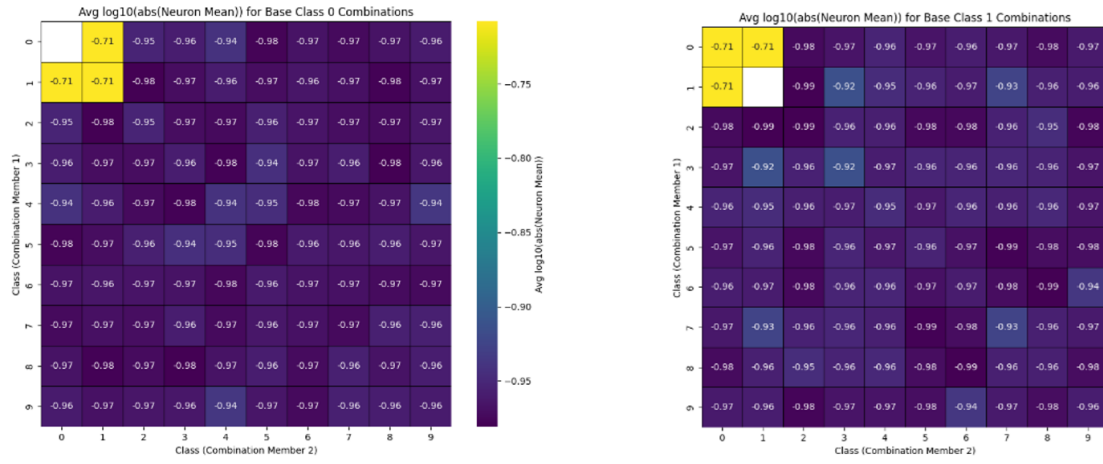
Figure 5. log mean of the average of the norm of neurons in Layer C0 (1st convolution) along the 40 epochs of training colored by the improvment of accuracy

DataFrame	Chunk	Mean	Std Dev	Chunk	Mean	Std Dev	Chunk	Mean	Std Dev	Chunk	Mean	Std Dev	Chunk	Mean	Std Dev
DF0	C0	-0.0297	0.4633	C1	-0.0026	0.2060	C2	0.0298	0.5299	D0	0.0122	0.4442	D1	-0.0544	0.7976
DF1	C0	-0.0297	0.4647	C1	-0.0028	0.2064	C2	0.0314	0.5251	D0	0.0124	0.4422	D1	-0.0543	0.7952
DF2	C0	-0.0288	0.4643	C1	-0.0029	0.2055	C2	0.0310	0.5219	D0	0.0127	0.4434	D1	-0.0532	0.7966
DF3	C0	-0.0283	0.4643	C1	-0.0024	0.2055	C2	0.0276	0.5244	D0	0.0116	0.4439	D1	-0.0556	0.7983
DF4	C0	-0.0317	0.4655	C1	-0.0031	0.2062	C2	0.0329	0.5212	D0	0.0121	0.4426	D1	-0.0560	0.7978
DF5	C0	-0.0303	0.4639	C1	-0.0025	0.2055	C2	0.0291	0.5230	D0	0.0124	0.4441	D1	-0.0552	0.7981
DF6	C0	-0.0296	0.4640	C1	-0.0028	0.2063	C2	0.0325	0.5233	D0	0.0125	0.4431	D1	-0.0548	0.7984
DF7	C0	-0.0314	0.4643	C1	-0.0023	0.2061	C2	0.0299	0.5248	D0	0.0121	0.4456	D1	-0.0542	0.7979
DF8	C0	-0.0297	0.4633	C1	-0.0026	0.2060	C2	0.0298	0.5299	D0	0.0122	0.4442	D1	-0.0544	0.7976
DF9	C0	-0.0297	0.4633	C1	-0.0026	0.2060	C2	0.0298	0.5298	D0	0.0122	0.4442	D1	-0.0544	0.7976

Figure 6. statistics about every layer

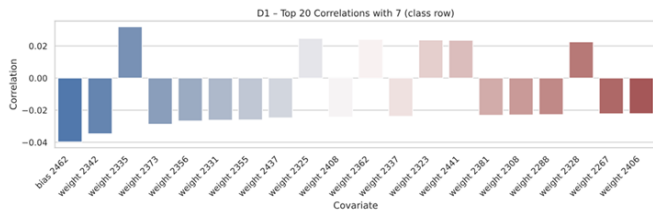


(a) heatmap models with 0 1 present kurtosis values

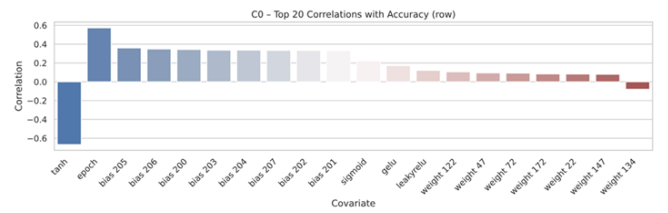


(b) heatmap models with 0 1 log10 of their absolute mean matrix norm value

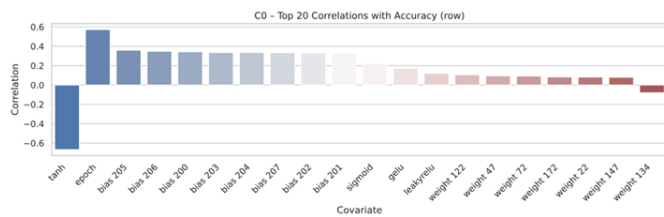
Figure 7. Statistical visualizations - Part 1



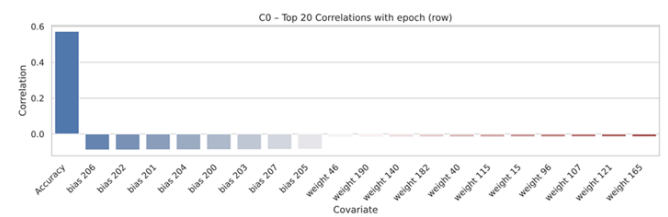
(a) 1st convolution C0 correlation matrix top 20 features



(b) 1st dense Layer D1 correlation matrix top 20 features

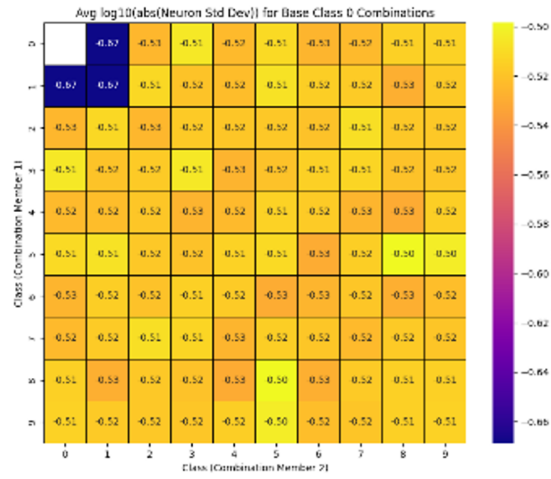


(c) features affecting accuracy most

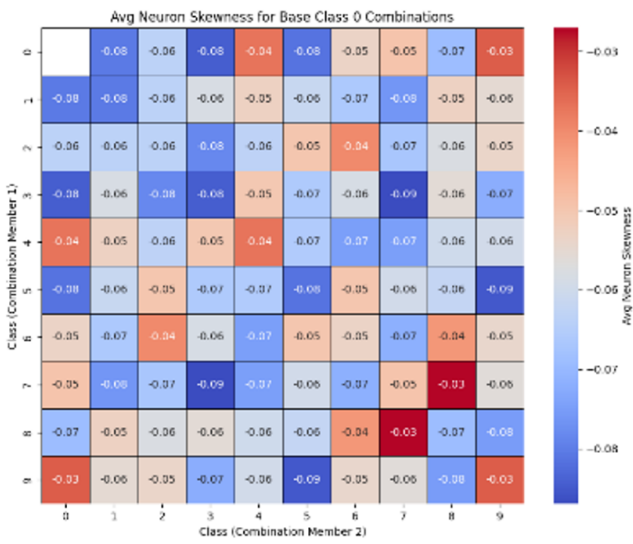


(d) features highly correlated with epoch evolution

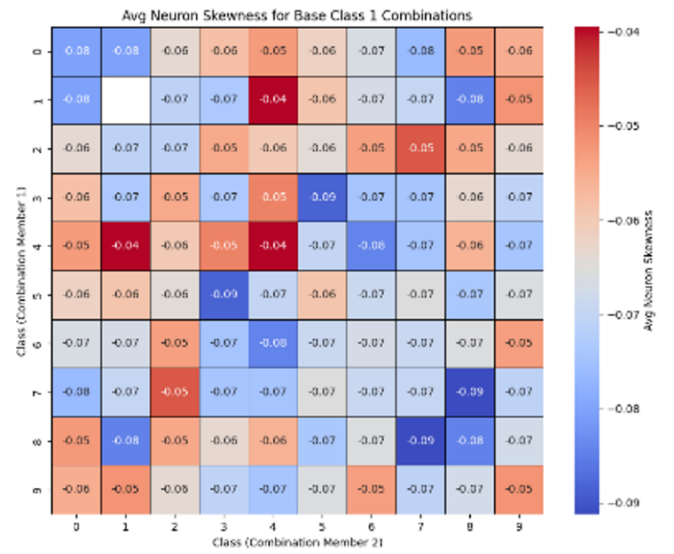
Figure 8. Correlation matrices comparison



(a) 0 std

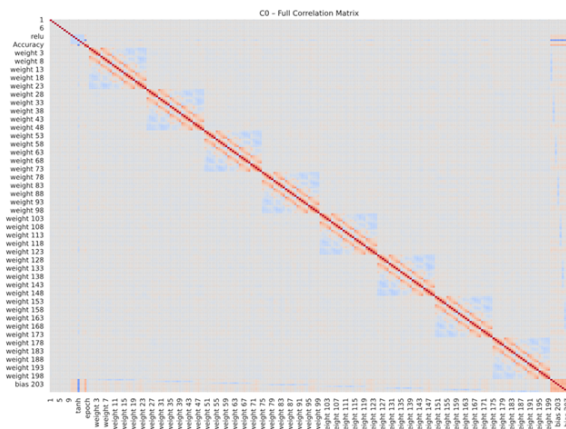


(b) 0 skewness

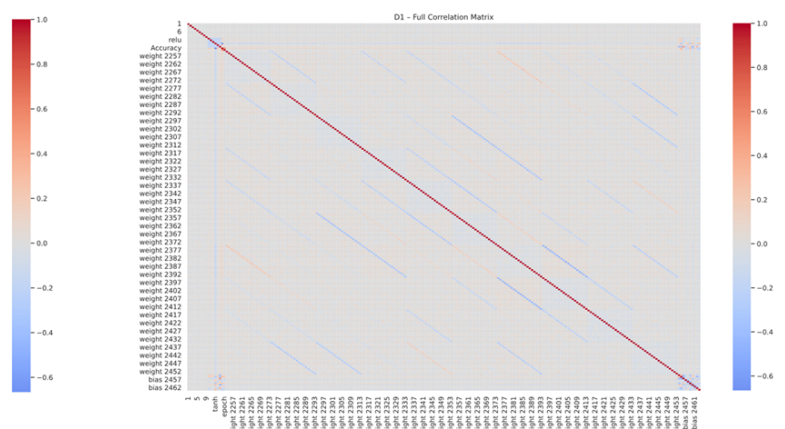


(c) 1 skewness

Figure 9. Statistical visualizations - Part 2



(a) C0 correlation matrix



(b) D1 correlation matrix

Figure 10. Correlation matrices comparison

Some activations are better early and stay better as training continues

we notice multiple regimes so time series segmentation is an idea. regression on each layer one by one is another (would need multiple models or 1-sequence model with input being the length of the largest CNN layer and we pad the rest of the sequence for smaller layers)

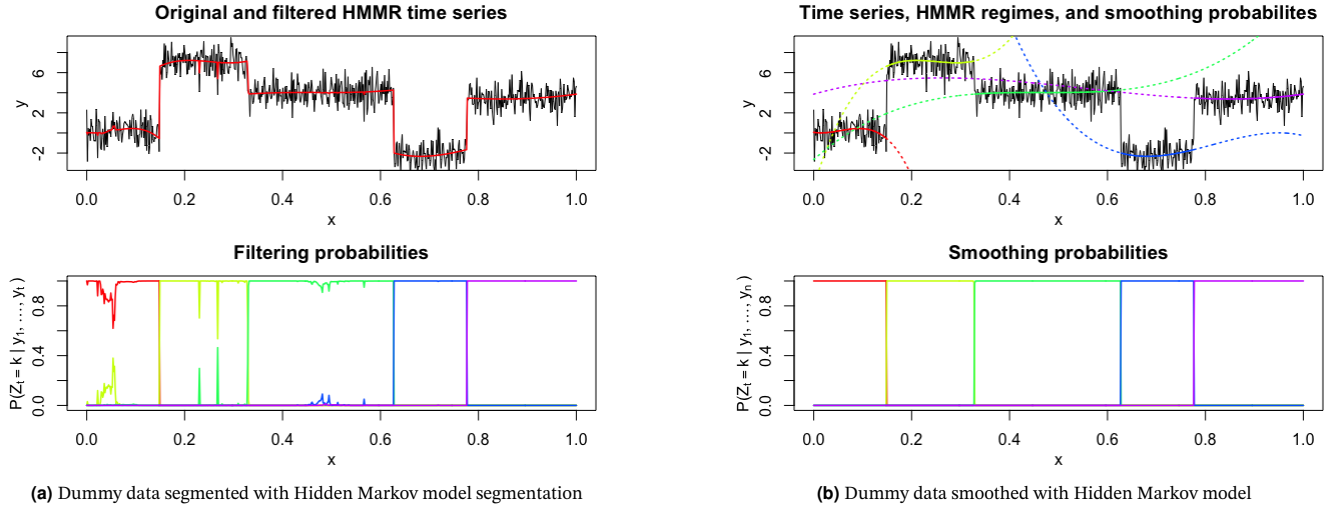


Figure 11. Examples of Time series segmentation

[8]
[86]

4. Meta-Learner Methodology:

4.1. Architecture:

model Argument	description	Our experiments	Hyper-representation paper
-	number of encoder	2	1
d_model	Dimensionality of model embeddings (input/output tokens).	960	972
N	Number of stacked encoder-decoder layers (blocks).	4	4
heads	Number of attention heads in each MultiHeadAttention layer.	4	12
d_ff	Hidden dimension in FeedForward networks (typically 4 d_model).	960	1140
neck	Latent/bottleneck dimension output of encoder, input to decoder.	512	700
dropout	Dropout probability applied after attention and FF layers.	0.07	0.01
max_seq_len	Max sequence length for positional encoding	50	50
epochs	Training epochs	800	1750
batch_size	batch size	36	500
-	data augmentation with neurone symmetry permutation	no	25000

4.2. Methodology:

we define the following :

- $\hat{y}_C$: is the ground truth label by a CNN
- $\hat{y}_C$: is the predicted label by a CNN
- $\hat{w}_{[g,i,e]}^{[l,Y_C]}$: are the ground truth weights of model from our zoo on I_C
- $\hat{w}_{[g,i,e]}^{[l,Y_C]}$: are the weights for a predicted CNN by the Transformer
- $\hat{w}_{[g,i,e]}^{[l,Y_C]}$: are the weights for a predicted CNN by the Transformer after finetuning on I_C
- $\hat{z}_t^{neck-1}$: is the latent representation after Encoder index $t \in [1, 2]$.
- $\hat{z}^{neck}$: is the latent representation after the neck dense layer.

The target task is to predict:

$$\text{CNN3}_{[g,i,e]}^{[Y_{C3}]}(\hat{w}_{[g,i,e]}^{[l,Y_{C3}]} | \hat{w}_{[g,i,e]}^{[l,Y_{C1}]} \cup \hat{w}_{[g,i,e]}^{[l,Y_{C2}]})$$

with weights obtained via:

$$\hat{w}_{[g,i,e]}^{[l,Y_{C3}]} = \text{Decoder}(\hat{z}^{neck}) = \text{Decoder}(\text{Dense}(\text{Encoder1}(\hat{z}_1^{neck-1}) \parallel \text{Encoder2}(\hat{z}_2^{neck-1}))) = \text{Decoder}(\text{Dense}(\text{Encoder1}(\hat{w}_{[g,i,e]}^{[l,Y_{C1}]}) \parallel \text{Encoder2}(\hat{w}_{[g,i,e]}^{[l,Y_{C2}]}))),$$

where $\parallel$ denotes concatenation, and $\hat{w}_{[g,i,e]}^{[l,Y_{C1}]}$, $\hat{w}_{[g,i,e]}^{[l,Y_{C2}]}$ are the ground truth weights of CNN1 and CNN2, respectively.

Given the pair:

- $\text{CNN1}_{[g,i,e]}^{[Y_{C1}]}([Y_{C1}] | I_{C1})$
- $\text{CNN2}_{[g,i,e]}^{[Y_{C2}]}([Y_{C2}] | I_{C2})$

the combined experience uses implicitly input union $I_{C1} \cup I_{C2}$ and outputs CNNs to be performance-checked on concatenation $[Y_{C1}, Y_{C2}] = Y_{C3}$ for their in-distribution metrics and $[0.9] \cdot Y_{C3}$ for out of distribution ones .

4.3. Tokenizing Numerical Values:

is done via a fixed dense layer ‘EmbedderNeuronGroup’

```

171 def get_clones(module, N):
172     return nn.ModuleList([copy.deepcopy(module) for i in range(N)])
173 class PositionalEncoder(nn.Module):
174     def __init__(self, d_model, max_seq_len=80, device='cuda'): #d"
175         super().__init__()
176         device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
177         self.d_model = d_model
178         self.device = device
179         # create constant 'pe' matrix with values dependant on
180         # pos and i
181         self.pe = self.generate_positional_encoding(max_seq_len, d_model)
182     def forward(self, x):
183         # make embeddings relatively larger
184         x = x * math.sqrt(self.d_model)
185         # add constant to embedding
186         seq_len = x.size(1)
187         # dynamically adjust positional encoding matrix based on sequence length
188         pe = self.pe[:, :seq_len]
189         pe.to(self.device)
190         x += pe
191         return x
192     def generate_positional_encoding(self, max_seq_len, d_model):
193         pe = torch.zeros(max_seq_len, d_model)
194         position = torch.arange(0, max_seq_len, dtype=torch.float).unsqueeze(1)
195         div_term = torch.exp(torch.arange(0, d_model, 2).float() * (-math.log(10000.0) /
196 d_model))
197         pe[:, 0::2] = torch.sin(position * div_term)
198         pe[:, 1::2] = torch.cos(position * div_term)
199         pe = pe.unsqueeze(0)
200         return pe
201     def attention(q, k, v, d_k, mask=None, dropout=None):
202         scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(d_k)
203         if mask is not None:
204             mask = mask.unsqueeze(1)
205             scores = scores.masked_fill(mask == 0, -1e9)
206         scores = F.softmax(scores, dim=-1)
207         if dropout is not None:
208             scores = dropout(scores)
209         output = torch.matmul(scores, v)
210         return output, scores
211 class MultiHeadAttention(nn.Module):
212     def __init__(self, heads, d_model, dropout=0.1):
213         super().__init__()
214         self.d_model = d_model
215         self.d_k = d_model // heads
216         self.h = heads
217         self.q_linear = nn.Linear(d_model, d_model)
218         self.v_linear = nn.Linear(d_model, d_model)
219         self.k_linear = nn.Linear(d_model, d_model)
220         self.dropout = nn.Dropout(dropout)
221         self.out = nn.Linear(d_model, d_model)
222     def forward(self, q, k, v, mask=None):
223         bs = q.size(0)
224         # perform linear operation and split into h heads
225         k = self.k_linear(k).view(bs, -1, self.h, self.d_k)
226         q = self.q_linear(q).view(bs, -1, self.h, self.d_k)
227         v = self.v_linear(v).view(bs, -1, self.h, self.d_k)
228         k = k.transpose(1, 2)
229         q = q.transpose(1, 2)
230         v = v.transpose(1, 2)
231         scores, sc = attention(q, k, v, self.d_k, mask, self.dropout)
232         output = self.out(torch.cat([sc, scores], dim=-1))
233         return output, sc
234 class EncoderLayer(nn.Module):
235     def __init__(self, d_model, heads, normalize=True, dropout=0.1, d_ff=2048):
236         super().__init__()
237         self.normalize = normalize
238         self.norm1 = Norm(d_model)
239         self.norm2 = Norm(d_model)
240         self.attn = MultiHeadAttention(heads, d_model, dropout=dropout)
241         self.ff = FeedForward(d_model, d_ff=d_ff, dropout=dropout)
242         self.dropout1 = nn.Dropout(dropout)
243         self.dropout2 = nn.Dropout(dropout)
244     def forward(self, x, mask):
245         if self.normalize:
246             x2 = self.norm1(x)
247         else:
248             x2 = x.clone()
249         res, sc = self.attn(x2, x2, x2, mask)
250         # x = x + self.dropout_1(self.attn(x2, x2, x2, mask))
251         x = x + self.dropout1(res)
252         if self.normalize:
253             x2 = self.norm2(x)
254         else:
255             x2 = x.clone()
256         res = self.ff(x2)
257         x = x + self.dropout2(res)
258         return x, sc
259     def return_x(self):
260         return x
261 class EmbedderNeuronGroup(nn.Module):
262     def __init__(self, d_model, seed=22):
263         super().__init__()
264         #print("EmbedderNeuronGroup")
265         self.neuron_l1 = nn.Linear(16, d_model) #24
266         self.neuron_l2 = nn.Linear(80, d_model) #26
267     def forward(self, x):
268         return self.multiLinear(x)
269     def multiLinear(self, v):
270         #print("Multi-Linear method", v.shape)
271         l = []
272         for ndx in range(26):
273             idx_start = ndx * 80
274             idx_end = idx_start + 80
275             l.append(self.neuron_l2(v[:, idx_start:idx_end]).clone())
276         # l2
277         for ndx in range(24):
278             idx_start = 26 * 80 + ndx * 16
279             idx_end = idx_start + 16
280             l.append(self.neuron_l1(v[:, idx_start:idx_end]).clone())
281         #print(len(l))
282         #print(len(l[0]))
283         final = torch.stack(l, dim=1)
284         # print(final.shape)
285         return final

```

(a) Helper functions and modules for Transformer

```

317 class SeqVec(nn.Module):
318     def __init__(self, d_model, max_seq_len):
319         super().__init__()
320         self.d_model = d_model
321         self.max_seq_len = max_seq_len
322         # Define linear layers
323         self.linear = nn.Linear(max_seq_len * d_model, 2464)
324     def forward(self, x):
325         batch_size = x.size(0)
326         x = x.view(batch_size, -1) # Flatten the sequence dimension
327         x = self.linear(x)
328         return x
329 class Neck2Seq(nn.Module):
330     def __init__(self, d_model, neck, max_seq_length):
331         super().__init__()
332         self.neurons = nn.ModuleList([nn.Linear(neck, d_model) for _ in
333 range(max_seq_length)])
334     def forward(self, x):
335         l = [neuron(x) for neuron in self.neurons]
336         final = torch.stack(l, dim=1)
337         return final
338 class DecoderNeuronGroup(nn.Module):
339     def __init__(self, d_model, N, heads, max_seq_len, dropout, d_ff, neck):
340         super().__init__()
341         self.N = N
342         self.embed = Neck2Seq(d_model, neck, max_seq_len)
343         self.pe = PositionalEncoder(d_model, max_seq_len)
344         #print("decoder drouous init, dropout")
345         self.layers = get_clones(EncoderLayer(d_model,
346 heads, normalize=True, dropout=dropout, d_ff=d_ff), N)
347         self.norm = Norm(d_model)
348         self.lay = SeqVec(d_model=d_model, max_seq_len=max_seq_len)
349     def forward(self, src, mask=None):
350         scores = []
351         x = self.embed(src)
352         x = self.pe(x)
353         for i in range(self.N):
354             x, sc = self.layers[i](x, mask)
355             scores.append(sc)
356         return self.lay(self.norm(x)), scores
357 class FeedForward(nn.Module):
358     def __init__(self, d_model, d_ff=2048, dropout=0.1):
359         super().__init__()
360         # We set d_ff as a default to 2048
361         self.linear_1 = nn.Linear(d_model, d_ff)
362         self.dropout = nn.Dropout(dropout)
363         self.linear_2 = nn.Linear(d_ff, d_model)
364     def forward(self, x):
365         x = self.dropout(F.relu(self.linear_1(x)))
366         x = self.linear_2(x)
367         return x
368 class Norm(nn.Module):
369     def __init__(self, d_model, eps=1e-6):
370         super().__init__()
371         self.size = d_model
372         # create two learnable parameters to calibrate normalisation
373         self.alpha = nn.Parameter(torch.ones(self.size))
374         self.bias = nn.Parameter(torch.zeros(self.size))
375         self.eps = eps
376     def forward(self, x):
377         norm = (
378             self.alpha * (x - x.mean(dim=-1, keepdim=True))
379             / (x.std(dim=-1, keepdim=True) + self.eps)
380             + self.bias)
381         return norm
382 class TransformerAE(nn.Module):
383     def __init__(self):
384         self.max_seq_len=10,
385         N=1,
386         heads=1,
387         d_model=100,
388         d_ff=100,
389         neck=20,
390         dropout=0.1,
391         **kwargs):
392         super().__init__()
393         self.N=N
394         self.heads=heads
395         self.dropout=dropout
396         self.d_ff=d_ff
397         self.d_model=d_model
398         self.max_seq_len=max_seq_len
399         self.neck=neck
400         self.enc1 = EncoderNeuronGroup(d_model=self.d_model, N=self.N, heads=self.heads, max_seq_len=self.max_seq_len,
401 dropout=self.dropout, d_ff=self.d_ff)
402         self.enc2 = EncoderNeuronGroup(d_model=self.d_model, N=self.N, heads=self.heads, max_seq_len=self.max_seq_len,
403 dropout=self.dropout, d_ff=self.d_ff)
404         self.dec = DecoderNeuronGroup(d_model=self.d_model, N=self.N, heads=self.heads, max_seq_len=self.max_seq_len,
405 dropout=self.dropout, d_ff=self.d_ff, neck=self.neck)
406         #print("Addition Approach")
407         self.vec2neck = nn.Linear(self.d_ff*2, self.neck)
408         #print("Stack Approach")
409         self.vec2neck = nn.Linear(2*self.d_ff * self.max_seq_len, self.neck)
410         self.tanh = nn.Tanh()
411         for p in self.parameters():
412             if p.dim() > 1:
413                 nn.init.xavier_uniform_(p)
414         if torch.cuda.is_available():
415             self.cuda()
416     def forward(self, inpl, inp2):
417         out1, scEnc1 = self.enc1(inpl) #21 and attentions score
418         out2, scEnc2 = self.enc2(inp2) #22 and attention score
419         out3=torch.cat([out1,out2], dim=2) #concatenation to get dimenson n_output=n_input*2
420         sum=torch.sum(out3, dim=1, keepdim=False)
421         vec2=self.vec2neck(sum) #fusion dense layer
422         tanh = nn.Tanh()
423         neck_t=tanh(vec2)
424         out, scDec = self.dec(neck_t) #predicted weights , decoder attention scores
425         return out, neck_t, scEnc1,scEnc2, scDec

```

(b) Decoder Layer and Transformer declaration

Figure 12. Transformer Architecture


```

encoder dropout init 0.07
encoder dropout init 0.07
decoder dropout init 0.07
=====
Layer (type:depth-idx)                   Param #
=====
TransformerAE
-EncoderNeuronGroup: 1-1                  --
  -EmbedderNeuronGroup: 2-1              --
    -Linear: 3-1                          16,320
    -Linear: 3-2                          77,760
    -PositionalEncoder: 2-2              --
    -ModuleList: 2-3                    --
      -EncoderLayer: 3-3                  5,530,200
      -EncoderLayer: 3-4                  5,530,200
      -EncoderLayer: 3-5                  5,530,200
      -EncoderLayer: 3-6                  5,530,200
    -Norm: 2-4                            1,920
  -EncoderNeuronGroup: 1-2              --
    -EmbedderNeuronGroup: 2-5            --
      -Linear: 3-7                        16,320
      -Linear: 3-8                        77,760
      -PositionalEncoder: 2-6            --
      -ModuleList: 2-7                  --
        -EncoderLayer: 3-9              5,530,200
        -EncoderLayer: 3-10             5,530,200
        -EncoderLayer: 3-11             5,530,200
        -EncoderLayer: 3-12             5,530,200
      -Norm: 2-8                          1,920
  -DecoderNeuronGroup: 1-3              --
    -Neck2Seq: 2-9                      --
      -ModuleList: 3-13                 24,624,000
      -PositionalEncoder: 2-10          --
      -ModuleList: 2-11                --
        -EncoderLayer: 3-14             5,530,200
        -EncoderLayer: 3-15             5,530,200
        -EncoderLayer: 3-16             5,530,200
        -EncoderLayer: 3-17             5,530,200
      -Norm: 2-12                          1,920
    -Seq2Vec: 2-13                      --
      -Linear: 3-18                     118,274,464
  -Linear: 1-4                          983,552
  -Tanh: 1-5                            --
=====
Total params: 210,546,336
Trainable params: 210,546,336
Non-trainable params: 0
=====
Tracking run with wandb version 0.22.1

```

Figure 13. number of parameters per layer

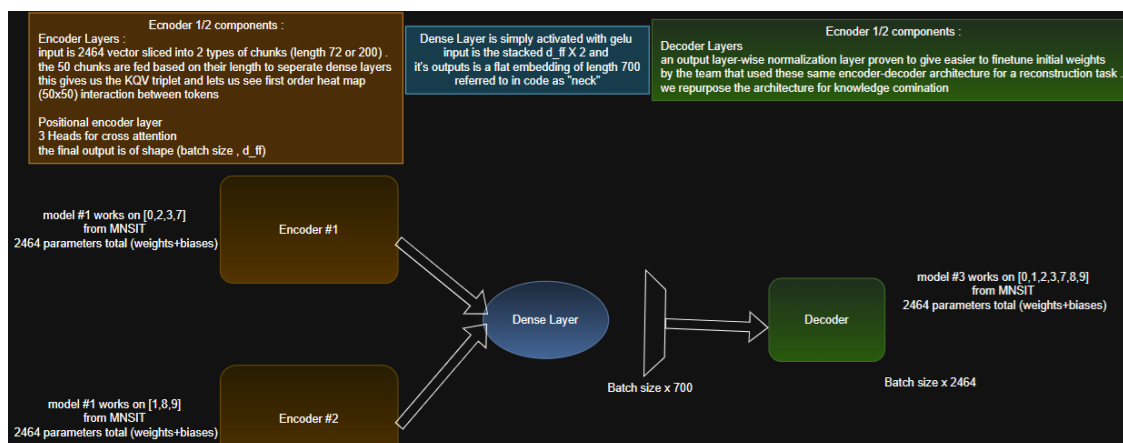


Figure 14. 2-encoder Transformer architecture for task merging

4.4. Training Loop Algorithm:

Algorithm 1: Training Procedure for Double-Encoder Transformer for CNN Weight Prediction

Input: Weights of two source CNNs (W_1, W_2), target CNN weights W_t , pretrained double-encoder Transformer T_θ

Output: Predicted target weights $\hat{W}_t$, performance diagnostics, and logged metrics

1 Initialization:

2 Initialize $track \leftarrow 0$, gradients $\nabla \leftarrow 0$

3 Create mixed-precision `Accelerator` with bfloat16 precision

4 Load list of scenarios $\{S_1, \dots, S_n\}$ from `./data/Scenario/`

5 **foreach** scenario S_t **do**

6 Load training, validation, and test pairs:

7 $train_pair2, val_pair2, test_pair2 \leftarrow np.load(S_t)$

8 Convert each pair to list of tensors (W_1, W_2, W_t)

9 Model setup:

10 Initialize encoders E_1, E_2 , transformer module T_θ , and decoder D

11 Initialize optimizers for encoder, decoder, and transformer

12 Move models to accelerator device and set them to training mode

13 Training Loop:

14 **for** $epoch = 1$ **to** N_{epochs} **do**

15 Shuffle $train_pair2$

16 **for** each batch (W_1^b, W_2^b, W_t^b) **do**

17 Encode: $z_1 \leftarrow E_1(W_1^b), z_2 \leftarrow E_2(W_2^b)$

18 Fuse latent codes: $z_f \leftarrow T_\theta(z_1, z_2)$

19 Decode: $\hat{W}_t^b \leftarrow D(z_f)$

20 Compute reconstruction loss $\mathcal{L}_{pred} = \|\hat{W}_t^b - W_t^b\|^2$

21 Compute auxiliary metrics (distances, eigenvalues, persistent homology graphs)

22 Combine metrics into total loss $\mathcal{L}_{total}$

23 Zero optimizer gradients

24 Backpropagate $\mathcal{L}_{total}$ and update model parameters

25 Clip gradients by norm to 1

26 Logging:

27 Log losses and metrics to Weights & Biases:

28 - $\mathcal{L}_{pred}, \mathcal{L}_{total}$

29 - Fisher / Wasserstein distances

30 - Eigenvalue spectra

31 - Mapper graph statistics and persistence diagrams

32 - Histograms of predicted vs. target weights

33 **end**

34 Optionally perform evaluation on validation pairs and save checkpoints

35 **end**

36 Compute test set predictions $\hat{W}_t$ and log final distances and graphs

37 **end**

38 **Output:** trained model T_θ and full W&B experiment log

4.5. Loss functions used and effects:

4.5.1. Q-quantile loss:

Measures distributional discrepancy at specific quantiles robust to outliers, sensitive to tail behavior.

$$\mathcal{L}_Q(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \frac{1}{N} \sum_{i=1}^N \rho_Q(w_i^{(1)} - w_i^{(2)}), \quad \rho_Q(u) = u(Q - \mathbb{I}_{\{u < 0\}}),$$

This loss is minimized when the Q -th quantile of the difference distribution is zero, i.e., the two CNNs align at that quantile.

4.5.2. Frobenius Norm Jacobian (Loss):

Measures overall sensitivity discrepancy between two models via Jacobian difference.

$$\mathcal{L}_{Frob} = \|\mathbf{J}^{(1)} - \mathbf{J}^{(2)}\|_F = \sqrt{\sum_{i=1}^C \sum_{j=1}^D (J_{ij}^{(1)} - J_{ij}^{(2)})^2},$$

Minimizing this loss encourages the two networks to have similar local input/output geometry.

4.5.3. Fisher Information Difference:

Measures intrinsic statistical dissimilarity between models via curvature of log-likelihood.

Compute the Fisher Information Distance (FID) between the two CNNs by treating their parameter vectors as points on a statistical manifold endowed with the Fisher information metric. This captures how differently the models encode information useful for comparing learning trajectories, robustness, or generalization.

$$\mathcal{D}_{Fisher}(\theta^{(1)}, \theta^{(2)}) = \inf_{\gamma} \int_0^1 \sqrt{\dot{\gamma}(t)^\top \mathcal{J}(\gamma(t)) \dot{\gamma}(t)} dt,$$

4.5.4. Contractive loss:

Penalizes sensitivity of hidden units to input encourages robust, invariant representations.

$$\mathcal{L}_{contractive} = \lambda \sum_{i=1}^H (h_i(1 - h_i))^2 \|\mathbf{w}_i\|_2^2 = \lambda \sum_{i=1}^H (h_i(1 - h_i))^2 \sum_{j=1}^D W_{ij}^2,$$

4.5.5. Wasserstein Distance/Geomloss:

Measures geometric discrepancy between distributions with smooth, differentiable approximation.

$$\mathcal{W}_{2,\varepsilon}^2(\mu, \nu) = \min_{\mathbf{P} \in \Pi(\mu, \nu)} \left\{ \sum_{i,j} P_{ij} \|x_i - y_j\|^2 - \varepsilon H(\mathbf{P}) \right\},$$

where

4.5.6. Difference in Norm of the vector:

Measures global magnitude discrepancy, simple, scale-sensitive, ignores direction.

$$\mathcal{L}_{\|\cdot\|} = \left| \|\mathbf{w}^{(1)}\|_2 - \|\mathbf{w}^{(2)}\|_2 \right| = \left| \sqrt{\sum_{i=1}^{2464} (w_i^{(1)})^2} - \sqrt{\sum_{i=1}^{2464} (w_i^{(2)})^2} \right|,$$

Minimizing this loss aligns the overall energy or scale of the two models, which can be a useful auxiliary objective in model compression, distillation, or weight-space interpolation.

4.5.7. Auto-regressive Loss:

Enforces sequential fidelity, later chunks penalized relative to earlier prediction accuracy.

$$\mathcal{L}_{\text{AR-MSE}} = \mathcal{L}_1 + \sum_{k=2}^K \lambda_{k-1} \frac{\mathcal{L}_k}{\mathcal{L}_{k-1} + \epsilon}, \quad \mathcal{L}_k = \frac{1}{B |J_k|} \sum_{b=1}^B \sum_{i \in J_k} (\hat{y}_i^{(b)} - y_i^{(b)})^2,$$

4.5.8. MAPE Loss:

$$\text{MAPE}(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \frac{100\%}{N} \sum_{i=1}^N \frac{|w_i^{(1)} - w_i^{(2)}|}{|w_i^{(2)}| + \epsilon}, \quad N = 2464,$$

4.5.9. Layer-wise-loss Normalization:

$$\mathcal{L}_{\text{LW-Norm}} = \sum_{\ell=1}^L \frac{1}{|\Theta_\ell|} \sum_{\theta \in \Theta_\ell} \ell(\theta^{(1)}, \theta^{(2)}) = \sum_{\ell=1}^L \frac{1}{|\Theta_\ell|} \|\theta_\ell^{(1)} - \theta_\ell^{(2)}\|_2^2,$$

4.5.10. Jensen-Shannon Loss:

$$\mathcal{L}_{\text{JS}}(\mathbf{p}, \mathbf{q}) = \sqrt{\text{JSD}(\mathbf{p} \parallel \mathbf{q})} = \sqrt{\frac{1}{2} D_{\text{KL}}(\mathbf{p} \parallel \mathbf{m}) + \frac{1}{2} D_{\text{KL}}(\mathbf{q} \parallel \mathbf{m})}, \quad \mathbf{m} = \frac{\mathbf{p} + \mathbf{q}}{2},$$

4.5.11. Fourier transform difference in Norm:

$$\mathcal{L}_{\text{FFT}}(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \frac{1}{N} \sum_{k=0}^{N-1} |\hat{w}_k^{(1)} - \hat{w}_k^{(2)}|, \quad \hat{\mathbf{w}} = \mathcal{F}(\mathbf{w}), \quad N = 2464,$$

4.5.12. Mel spectrogram Difference:

$$\mathcal{L}_{\text{Mel}}(\mathbf{w}^{(1)}, \mathbf{w}^{(2)}) = \|\mathbf{M}^{(1)} - \mathbf{M}^{(2)}\|_F = \sqrt{\sum_{t=1}^T \sum_{m=1}^M (M_{t,m}^{(1)} - M_{t,m}^{(2)})^2},$$

4.5.13. Mel Spectrogram Frechet inception distance:

$$\text{MS-FID} = \|\mu_1 - \mu_2\|_2^2 + \text{Tr}(\Sigma_1 + \Sigma_2 - 2(\Sigma_1^{1/2} \Sigma_2 \Sigma_1^{1/2})^{1/2}),$$

4.5.14. Gromov-wasserstein loss:

$$\text{GW}_p^p(\mu, \nu) = \min_{T \in \Pi(\mu, \nu)} \sum_{i,j=1}^N \sum_{k,\ell=1}^N |d_X(x_i, x_k) - d_Y(y_j, y_\ell)|^p T_{ij} T_{k\ell},$$

$$\text{GW}_{p,\epsilon} = \min_{T \in \Pi(\mu, \nu)} \langle \mathbf{L}, \mathbf{T} \otimes \mathbf{T} \rangle - \epsilon H(\mathbf{T}),$$

Why it matters for CNN weights:

Invariant to permutation of neurons/channels (no need for explicit matching). Captures global structure: e.g., whether both models have a few large-magnitude weights surrounded by small ones. Suitable for comparing spectral embeddings or topological summaries derived from weight vectors.

4.5.15. Bottleneck distance loss:

Measures similarity of topological features (persistence diagrams) via worstcase matching cost.

$$d_B(\mathcal{D}_1, \mathcal{D}_2) = \inf_{\gamma \in \Gamma(\mathcal{D}_1, \mathcal{D}_2)} \sup_{p \in \mathcal{D}_1} \|p - \gamma(p)\|_\infty,$$

4.5.16. Latent:

Measures consistency of latent geometry before and after fusion via a shared dense bottleneck.

Explanation of the Loss

Intra-branch alignment: The predictions (z1_pred, z2_pred) are pulled toward the target embedding (z1_tg, z2_tg) via MSE.

This encourages:

Each encoder to faithfully encode its input, The prediction (from a combined or intermediate representation) to be closer to the target than the raw inputs,

The merged bottleneck to align with a desired fused representation.

$$\mathcal{L}_{\text{total}} = \sum_{k=1}^2 \mathcal{L}_Z^{(k)} + \sum_{k=1}^2 \frac{2 \mathcal{L}_{\text{pred}}^{(k)}}{\mathcal{L}_{x_1}^{(k)} + \mathcal{L}_{x_2}^{(k)}}.$$

Why It's Useful:

Latent consistency: Ensures that the geometry of embeddings is preserved through the fusion operation. Relative supervision: The ratio term prevents trivial collapse if inputs already match the target well, the model isn't forced to overfit the prediction. Dual-branch validation: By enforcing the same target output [1] for both merged branches, you encourage cross-branch agreement in the final representation.

This design is well-suited for self-supervised fusion, multi-view learning, or cross-modal alignment where you want to verify that merging latent codes doesn't distort semantic content.

4.5.17. multipersistence L2 :

4.5.18. Time-Series segmentation guided normalisation loss :

4.6. Other Types of Attention

Differences Between This Implementation and Classical Multi-Head Attention

The MultiHeadAttention class in this code is a close implementation of the classical multi-head attention mechanism as described in the original Transformer paper ("Attention Is All You Need" by Vaswani et al., 2017). It computes scaled dot-product attention in parallel across multiple heads, concatenates the results, and applies a final linear projection. However, there are a few minor differences in design, functionality, and return values. I'll break them down step by step:

- Core Computation (Scaled Dot-Product Attention):

Classical: The attention for each head is computed as $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$, where Q, K, V are linearly projected versions of the inputs, and d_k is the dimension per head (typically d_{model}/h , with h being the number of heads). The softmax is applied along the last dimension.

This Implementation: Identical. The attention function computes exactly this: $\text{scores} = \text{torch.matmul}(q, k.\text{transpose}(-2, -1)) / \text{math.sqrt}(d_k)$, followed by masking (if provided), softmax, optional dropout, and $\text{torch.matmul}(\text{scores}, v)$. This is the standard scaled dot-product.

- **Linear Projections:**
 Classical: Separate linear layers project the input to queries (Q), keys (K), and values (V), each from d_{model} to $h \times d_k$ (i.e., the full multi-head projection). After per-head attention, outputs are concatenated and projected back via another linear layer from $h \times d_k$ to d_{model} .
 This Implementation: Matches exactly. It uses q_{linear} , k_{linear} , and v_{linear} (each $d_{\text{model}} \rightarrow d_{\text{model}}$), reshapes to split into heads (via `.view(bs, -1, self.h, self.d_k)` and transpose), computes attention per head implicitly in parallel, concatenates with `.transpose(1, 2).contiguous().view(bs, -1, self.d_model)`, and applies self.out (another linear $d_{\text{model}} \rightarrow d_{\text{model}}$).
- **Masking:**
 Classical: Supports optional masking (e.g., for padding or future tokens in decoders) by setting masked positions to a large negative value before softmax.
 This Implementation: Identical. If a mask is provided, it's unsqueezed and used in `masked_fill(mask == 0, -1e9)` of this core variable, which is the standard approach.
- **Dropout:**
 Classical: Dropout is typically applied after the softmax (on the attention probabilities) and also in other parts of the Transformer (e.g., feed-forward layers). The original paper uses a dropout rate of 0.1 by default.
 This Implementation: Matches. Dropout is applied post-softmax in the attention function (`scores = dropout(scores)` if provided). The class initializes with `dropout=0.1`, and it's passed to the attention function.
- **Return Values:**
 Classical: Most standard implementations (e.g., in PyTorch's `nn.MultiheadAttention`) return only the attention output (the weighted values). Attention weights (probabilities) are optional and often require a flag like `'return attn weights'=True`.
 This Implementation: A key difference—it always returns both the output and the attention weights (return output, sc, where sc is the post-softmax scores from the attention function). This is useful for interpretability or visualization (e.g., in the `EncoderLayer`, which propagates these scores). Classical implementations often omit weights by default to save computation/memory, but this is a minor extension, not a fundamental change.
- **Input Flexibility:**
 Classical: Designed to handle queries (Q), keys (K), and values (V) that can be from different sources (e.g., self-attention when $Q=K=V$, or cross-attention when Q differs).
 This Implementation: Fully general—accepts separate q, k, v tensors, allowing for both self- and cross-attention. In the code's usage (e.g., in `EncoderLayer`), it's called with `self.attn(x2, x2, x2, mask)` for self-attention, which is standard.
- **Efficiency and Assumptions:**
 Classical: Assumes batched inputs with shape ('batch size', 'seq len', 'd model'). Head splitting is done via reshaping, which is efficient on GPUs.
 This Implementation: Identical assumptions and efficiency. It uses `.view()` and `.transpose(1, 2)` for head-parallel computation, which is the optimized PyTorch way.
- **Initialization and Other Details:**
 Classical: No specific initialization mentioned in the paper, but common practice is Xavier uniform for weights.
 This Implementation: In the parent `TransformerAE`, it applies `'nn.init.xavier_uniform(p)'` for all parameters with >1 dimension, which aligns with best practices but isn't a difference from classical (many libs do this).
 Overall Summary of Differences: This is essentially the classical multi-head attention with one small addition—always returning the attention weights (sc) alongside the output. This makes it slightly more verbose for debugging or analysis but doesn't change the math or behavior. If you strip out the sc return, it's identical to reference implementations like Harvard's Annotated Transformer or PyTorch's official module (minus some optional features like bias terms or in-proj optimizations in PyTorch's version).

Comparison to Other Types of Attention Mechanisms

Attention mechanisms have evolved significantly since the original additive attention in seq2seq models. Below, I'll compare the multi-head attention (MHA) from this code (which is classical scaled dot-product MHA) to other major variants. I'll categorize them and highlight key differences in computation, use cases, efficiency, and limitations. This is not exhaustive but covers the most influential types.

1. Single-Head vs. Multi-Head Attention

[33]

[59]

[30]

Single-Head: Computes attention once over the full embedding dimension (e.g., basic scaled dot-product without head splitting).

Comparison to This MHA: Single-head is a special case of MHA with `heads=1`. MHA (as here) allows learning diverse relationships by projecting into multiple subspaces, improving representation power (e.g., capturing different syntactic/semantic aspects). This implementation supports arbitrary heads, making it more flexible. Single-head is simpler/faster but often underperforms on complex tasks like NLP or vision.

2. Additive (Bahdanau) Attention [3]

Description: From Bahdanau et al. (2014) for seq2seq. Scores are computed as $score = v^T \tanh(W_1 q + W_2 k)$, an additive (MLP-based) alignment rather than dot-product.

Comparison to This MHA: Additive is slower ($O(d)$ per pair due to the MLP) and less scalable than dot-product ($O(1)$ per pair after matmul). This MHA uses efficient scaled dot-product, which parallelizes better on GPUs. Additive works well for smaller models/sequences but is largely replaced by dot-product in Transformers due to speed. No multi-head in classic additive.

3. Dot-Product Attention (Without Scaling)

Description: Similar to scaled dot-product but without $1/\sqrt{d_k}$ (e.g., early versions in [2]).

Comparison to This MHA: This implementation includes scaling to prevent softmax saturation for large d_k , as in the Transformer paper. Unscaled can lead to vanishing gradients; scaling stabilizes training. Otherwise, very similar—both rely on QK^T .

4. Self-Attention vs. Cross-Attention

Self-Attention: Q, K, V from the same sequence (e.g., modeling intra-sequence dependencies).

Cross-Attention: Q from one sequence (e.g., decoder), K/V from another (e.g., encoder outputs).

Comparison to This MHA: This class supports both (via separate q/k/v args), but in the code, it's used as self-attention ($q=k=v$). Cross-attention is identical in math but applied differently in architectures like decoder layers. No structural difference here.

5. Local/Sliding Window Attention [75]

Description: Restricts attention to a fixed window around each token (e.g., in Longformer or Image Transformer) to reduce $O(n^2)$ complexity to $O(n \times w)$, where w is window size.

Comparison to This MHA: This is global (full $O(n^2)$ over sequence length), attending to all positions. Local is more efficient for long sequences but loses global context. This implementation could be adapted with a custom mask for locality, but it's not built-in.

6. Sparse Attention (e.g., Sparse Transformer, Reformer)[13]

Description: Uses sparsity patterns (e.g., fixed strides or hashing) to attend only to a subset of positions, achieving $O(n \log n)$ or better complexity.

Comparison to This MHA: This is dense/full attention (computes all pairs). Sparse is more scalable for very long inputs (e.g., $>10k$ tokens) but may miss some interactions. Reformer's LSH (locality-sensitive hashing) approximates attention; this exact computation is more accurate but quadratic.

7. Linear/Efficient Attention (e.g., Linformer, Performer)[85]

Description: Approximates full attention with low-rank projections or kernel tricks (e.g., Performer uses random Fourier features for softmax approximation), reducing complexity from $O(n^2)$ to $O(n)$.

Comparison to This MHA: This is exact quadratic attention, which is precise but memory-intensive. Linear variants trade a bit of accuracy for massive speedups on long sequences. For example, Linformer projects K/V to lower dims before matmul; this doesn't.

8. Grouped Query Attention (GQA) [52] or Multi-Query Attention (MQA) [82]

Description: In large models like LLaMA or GPT, shares K/V projections across groups of queries (GQA) or all heads (MQA) to reduce KV cache size in inference.

Comparison to This MHA: This uses separate projections per head for Q/K/V (full MHA). GQA/MQA are optimizations for deployment—faster inference with similar quality but less parallel diversity in heads. Not applicable here since this is a small custom model.

9. Relative Positional Attention (e.g., Transformer-XL, DeBERTa) [11]

Description: Adds relative position biases to scores (e.g., $QK^T + QR^T$, where R is relative positions) for better handling of long-range dependencies.

Comparison to This MHA: This uses absolute positional encodings (via PositionalEncoder) but no relative biases in attention. Relative is better for extrapolation to longer sequences; this might struggle with position shifts.

10. Flash Attention or Optimized Variants [71]

Description: Hardware-aware optimizations (e.g., DAO's FlashAttention) fuse softmax/matmul/kernel ops for faster training/inference without changing math.

Comparison to This MHA: This is a naive PyTorch implementation (separate matmuls), which is correct but slower on GPUs due to memory access. Flash is drop-in compatible but requires custom kernels—not a functional difference, just efficiency.

General Trade-offs Across Types:

- **Efficiency:** This classical MHA is $O(n^2d)$ time/space—great for short sequences (like here, 'max seq len'=80) but scales poorly. Efficient variants (linear/sparse) are for long inputs.
- **Expressivity:** Exact dot-product (as here) is highly expressive; approximations (e.g., Performer) may lose nuance.
- **Use Cases:** This suits standard Transformers for seq modeling. Additive for older RNNs; sparse/linear for genomics/videos; GQA for LLMs.

When to Choose This: If you need vanilla Transformer behavior with attention weights exposed, this is perfect. For production, consider optimized libs like Hugging Face Transformers.

4.7. Other Types of Transformers :

- **Mamba**[61]: Linear scaling with sequence length, constant memory, 5x faster throughput than Transformers, handles million-token contexts via selective state space models (SSMs) without full attention.
- **Jamba**[65]: Hybrid Transformer-Mamba MoE model with 52B parameters, processes 256k tokens on a single GPU using 4GB KV cache, balances efficiency and reasoning.
- **cosFormer**[50]: Replaces softmax with cosine-based reweighting for linear attention, achieves 10x memory reduction and 2-22x speed-up while retaining 92-97% of Transformer accuracy.
- **Performer**[42]: Approximates softmax attention using random feature maps, reduces complexity to linear time, 4,000x faster on long sequences with 92-97% accuracy preservation.
- **Linformer**[24]: Projects attention keys/values to lower dimension for linear complexity, 76% memory savings, achieves 99% of RoBERTa performance.
- **FNet**[45]: Substitutes self-attention with Fourier Transforms for token mixing, 80% faster training, low memory, suitable for speed-critical tasks.
- **Reformer**[21]: Employs locality-sensitive hashing (LSH) and reversible layers, 10x memory reduction, processes up to 64k tokens efficiently.
- **BigBird**[37]: Sparse attention with random, window, and global tokens, linear complexity, handles 8x longer sequences than BERT with theoretical completeness.
- **Longformer**[19]: Uses sliding window attention plus global tokens, scales to 4,096 tokens linearly, drop-in replacement for BERT.
- **Switch Transformer** [43]: Mixture-of-Experts (MoE) with top-1 routing, scales to 1.6T parameters with 7x training speedup and constant compute.
- **Griffin** [62]: Hybrid of gated linear recurrences (Hawk-like) and local attention, combines recurrence for efficiency with attention for reasoning.
- **Mamba-Transformer Hybrids** [63]: Integrates Mamba's state space efficiency with Transformer's attention for reasoning, achieves comparable quality with better memory utilization.

5. Appendix:

5.1. Continual Learning Primer

[89]
[87]
[74]
[36]
[48]
[7]
[83]
[57]
[73]
[4]
[31]

5.2. Hyper representation Current results

populations [68]
sampling [77]
SOTA [72]
[32]
[51]
[78]

Table 1. Performance of the dataset’s conditional sampling on 10 unseen real-world datasets. We report the averaged accuracy on ten unseen test datasets over 3 different runs fine-tuned for 50 epochs. **pret(imnet)**: pretrained on ImageNet1k. Bold numbers indicate the best result in each row.

Datasets	No fine-tuning			50 epochs Fine-Tuning			# of classes
	Random init.	pret(imnet)	D2NWG(current SOTA)	Random init.	pret(imnet)	D2NWG(current SOTA)	
Gemstones	1.13±0.52	0.62±0.00	1.86±0.25	70.59±0.91	67.49±0.43	76.06±0.88	87
Dog Breeds	0.55±0.22	0.69±0.00	1.87±0.39	80.78±0.28	78.13±0.49	80.68±0.88	133
Dessert	21.03±2.44	12.50±0.00	99.40±0.02	95.83±0.34	94.64±0.00	99.40±0.02	5
Colorectal Histology	11.77±2.88	11.00±0.00	18.12±0.25	90.34±0.33	89.75±0.19	93.65±0.10	8
Drawing	10.86±1.22	11.00±0.00	11.87±0.93	90.20±0.16	90.00±0.16	89.00±0.16	10
Alien vs Predator	51.48±2.09	28.88±0.00	78.15±0.52	98.52±0.52	98.89±1.42	97.77±0.00	2
COVID-19	20.13±18.66	46.53±0.00	47.22±0.00	93.86±0.16	93.40±0.49	94.56±0.71	3
honey-bee-pollen	49.54±1.30	50.00±0.00	56.94±4.53	93.05±0.00	88.89±0.00	93.55±4.53	2
Speed Limit Signs	30.55±2.27	25.00±0.00	31.48±10.23	83.33±0.00	86.11±0.00	90.74±1.31	4
Japanese Characters	0.03±0.00	0.08±0.00	0.50±0.22	53.17±0.15	62.33±0.16	62.16±0.47	1566

5.3. Optimal Transport Primer

[1]
[81]
[94]
[88]

$$\det(D^2u(x)) = \frac{f(x)}{g(\nabla u(x))}, \quad x \in \Omega$$

$D^2u(x)$: Hessian of u (must be positive definite for strict convexity). $\nabla u(x)$: Gradient (the transport map is ∇u). $f(x)$: Source density. $g(y)$: Target density (evaluated at $y = \nabla u(x)$). The equation enforces the Jacobian condition: the pushforward of f via ∇u equals g .

Optimal Transport (OT) is a mathematical framework fundamentally concerned with defining the most efficient way to move mass from a starting distribution to a target distribution, given a defined cost function for transportation. The sources describe OT through its historical formulations, its relaxed version, the resulting distance metrics, and its crucial applications in modern computational problems.

Part 1: Explanation of Optimal Transport

- **Historical Foundations (Monge and Kantorovich)** Transportation theory, the study of optimal transportation and allocation of resources, was first formalized by the French mathematician Gaspard Monge in 1781. The core challenge was to find an optimal plan for moving resources (like dirt or ore) from multiple supply points (mines) to multiple demand points (factories). Major theoretical advances were later made by the Soviet mathematician and economist Leonid Kantorovich in the 1940s. Consequently, the problem is often known as the Monge Kantorovich transportation problem.
- **Monge’s Formulation (MP)** Monge’s original problem attempts to find a one-to-one measure-preserving map T (or s) from a source space X with measure α to a target space Y with measure β . The goal is to minimize the total transportation cost $\inf_T \int_X c(x, T(x)) d\alpha(x)$ where $c(x, T(x))$ is the cost of moving a unit of mass from location x to $T(x)$, and $T_\# \alpha = \beta$ means that T pushes the mass distribution α forward exactly onto β . Limitation: The Monge formulation can be ill-posed because an optimal map T might fail to exist. This happens, for example, if the source measure α is a single Dirac mass but the target measure β is not, as a single point cannot be deterministically mapped to multiple locations.
- **Kantorovich’s Formulation (MKP):** The Relaxation Kantorovich introduced a relaxed, probabilistic version (MKP) which avoids the need for a deterministic map T . Instead of finding a single mapping, MKP seeks an optimal coupling, π , which is a joint probability measure on $X \times Y$. The key idea is that mass at a source point x can be “split” and potentially dispatched across several target locations y . This flexibility is captured by $\pi(x, y)$, which describes the amount of mass flowing from x to y . The objective is to minimize the generalized transport cost $\int_{X \times Y} c(x, y) d\pi(x, y)$, subject to constraints ensuring that π ’s marginals are the original distributions α and β .

Advantages: This relaxation solves the issues of existence and symmetry inherent in Monge’s problem. The continuous Kantorovich problem always has a solution on compact domains. Critically, this formulation constitutes an infinite-dimensional linear program.

Probabilistic Interpretation: If $(X, Y) \sim \pi$, the marginal constraint means that $X \sim \alpha$ and $Y \sim \beta$. The problem minimizes the expected cost $\mathbb{E}(c(X, Y))$ over all joint distributions π with the correct marginals.

- The Resulting Metric: Wasserstein Distance (W_p) When the cost function $c(x, y)$ is defined as a power of the distance between points, $c(x, y) = d(x, y)^p$, the minimum cost attained by the Kantorovich problem defines the p -Wasserstein distance ($W_p(\alpha, \beta)$). This metric quantifies the cost of transforming one distribution (α) into another (β). $W_p(\alpha, \beta) = (\min_{\pi \in U(\alpha, \beta)} \int_{X \times Y} d(x, y)^p d\pi(x, y))^{1/p}$. The Wasserstein distance (also known as the optimal transport distance or Kantorovich Rubinstein norm) is a metric (it is symmetric, positive, and satisfies the triangle inequality). It is particularly useful because it is a weak distance, meaning it can compare singular distributions (like discrete point clouds) and quantify spatial shift between their supports.
- Duality and Monge-Kantorovich Equivalence The Kantorovich problem, being a linear program, possesses a dual formulation that maximizes a term involving Kantorovich potentials (f and g). A fundamental result linking the two formulations is Brenier's Theorem. For the quadratic cost ($c(x, y) = \|x - y\|^2$), and assuming the source measure α has a density, the unique optimal transport map T (Monge's solution) exists and is the gradient of a convex function, ϕ ($T = \nabla \phi$). This implies that the Monge and Kantorovich costs are the same, and the optimal coupling π is induced by this deterministic map: $\pi = (\text{Id}, T)_\# \alpha$.

In this notation we are studying

$$\int \phi(y) d\mu(y) + \int \psi(x) d\nu(x)$$

under the constraints $\{\phi(x) + \psi(y) \geq c(x, y)\}$.

This is a dual problem $\iff$ "this is a quadratic cost Optimal transport problem" between μ and ν . We are solving

$$\min \int |x - T(x)|^2 d\mu(x) \quad \text{under the constraints } T_\#(\mu) = \nu.$$

So what's the role of s and $\nabla \psi$?

$$r(x) = \nabla \psi(s(x)), \quad s(x) = \nabla \phi(r(x)).$$

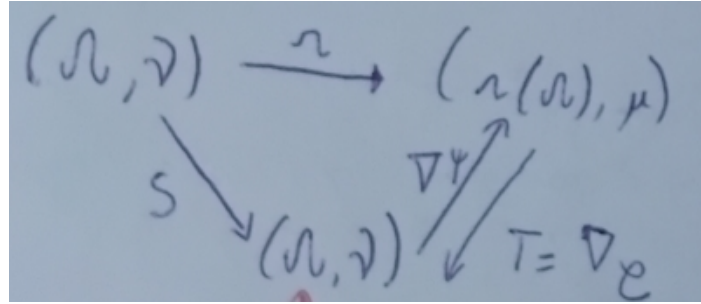


Figure 15. Polar factorization allows a rearrangement of the source distribution to find a unique optimal map

1. S is a **measure-preserving rearrangement**
2. $S(x) = \nabla \phi(x)$ almost everywhere
3. $S(\cdot) = \nabla \phi(\cdot)$

This is precisely the **polar factorization** of $S \in L^2(\mu, \nu)$:

$$S = \nabla \phi \circ S^p \quad (\text{Brenier}) \quad \text{or} \quad S = S^p \circ \nabla \phi \quad (\text{McCann, polar factorization})$$

where $\nabla \phi : \text{supp}(\mu) \rightarrow \text{supp}(\nu)$ is the optimal transport map and S^p is the (unique) measure-preserving part of S .

- Computation and Regularization (Sinkhorn) Since the exact OT problem is often computationally expensive, especially in high dimensions, a common approach is Entropic Regularization. This involves modifying the Kantorovich objective function by adding a scaled Kullback-Leibler (KL) divergence term (entropy) as a regularizer:

$L_\epsilon^c(\alpha, \beta) := \min_{\pi \in U(\alpha, \beta)} \int_{X \times Y} c(x, y) d\pi(x, y) + \epsilon \text{KL}(\pi | \alpha \otimes \beta)$ This makes the problem strictly convex, guaranteeing a unique, smooth solution P_ϵ . The solution P_ϵ can be efficiently computed using Sinkhorn's algorithm, which is derived from the dual regularized problem.

Part 2: Concepts to Retain from the Sources The sources emphasize that Optimal Transport is a crucial concept in contemporary machine learning for introducing geometric knowledge and defining robust metrics, particularly when dealing with structured data like neural network parameters or topological descriptors.

1. The Wasserstein Distance as a Topological Metric

The Wasserstein distance (W_q) is a fundamental metric used to compare persistence diagrams (PDs) in Topological Data Analysis (TDA). This distance leverages optimal transport theory to calculate the cost of matching features (points) between two diagrams.

When used in optimization pipelines, the Sliced Wasserstein Distance (SWD) or related kernelized versions are often preferred because the full Wasserstein distance involves combinatorial matching and is generally non-differentiable.

The Optimal Transport distance is used to define distances between discrete signed measures, such as the Hilbert decomposition signed measure derived from multiparameter persistence, enabling the creation of differentiable topological losses.

2. Optimal Transport for Neural Network Structure (OTMANN)

The concept of optimal transport is adapted to create the OTMANN (Optimal Transport Metrics for Architectures of Neural Networks) distance.

OTMANN is formulated as an Optimal Transport program that minimizes a matching scheme to quantify the dissimilarity between two network architectures based on structural position and layer properties.

This metric is used within the NASBOT (Neural Architecture Search with Bayesian Optimisation and Optimal Transport) framework, where a measure of similarity (a kernel) is derived from the OTMANN distance to facilitate Bayesian optimization.

Although OTMANN uses an OT formulation, it is specifically noted that it is not a Wasserstein distance because the underlying mass supports and cost matrices change based on the specific networks being compared.

3. Role in Loss Functions and Continual Learning

Optimal Transport provides a robust way to compare distributions of complex objects. This is highly relevant to my objective, as you noted that the Wasserstein distance (WD) decreases meaningfully during fine-tuning of weight hyper-representations, suggesting it captures functional similarity better than simple L_2 losses.

Optimal Transport is seen as a key component for designing loss functions that can preserve both the shape (Topological manifold) and content of data, making it a prime candidate for optimizing knowledge combination in generative models like the one you are developing.

OT finds applications in analyzing generative models via Flow Matching, where an optimal transport map is sought to transform a reference distribution (like Gaussian noise) to a target data distribution.

5.4. Persistent Homology Primer

This section outlines the Persistent Homology (PH) primer and concludes with a recommendation for a suitable loss function mixture for my Transformer model, specifically designed to rival diffusion-based methods like D2NMG.

[76]

My Transformer model, utilizing two encoders for Convolutional Neural Network (CNN) weights (from MNIST sub-partitions) and a dense layer merging embeddings, feeding a decoder for the union CNN, acts as a generative hyper-representation model. This framework seeks to predict functional weights, thus engaging in weight regression and knowledge aggregation for continual learning.

The generative hyper-representation approach (like D2NMG) leverages latent diffusion to harness knowledge from a collection of pretrained weights, generating dataset- or task-conditioned parameters. Critically, diffusion models for weight generation, such as D2NMG, model the parameter distribution using a VAE encoder/decoder structure and a diffusion process, optimizing the Latent Diffusion Model (LLDM) objective, which is an L_2 loss on the noise component ϵ .

My observed challenge—that Mean Squared Error (MSE) is inadequate and the Wasserstein Distance (WD) decreases meaningfully during fine-tuning—suggests that optimizing solely for reconstruction precision (like L_{MSE} or the L_2 inherent in the LLDM objective) is insufficient. This strongly motivates using a topologically-aware loss that enforces manifold structure, aiding in finding weights that lie on the optimal solution surface.

5.4.1. Topological Data Analysis (TDA) Background

[60]

Persistent Homology and Classical TDA Topological Data Analysis (TDA) is a field of algebraic topology aimed at providing quantifiable, comparable, robust, and concise summaries of the shape of data. PH is the core tool of TDA, analyzing the evolution of topological features across different scales. The process begins by representing the data space as a simplicial complex. A filtration is then constructed: a nested sequence of increasing subsets of the simplicial complex. Homology provides the mathematical language for counting the holes, or cavities, in a topological space. The dimension of the i -th homology group, $H_i(K)$, is the i -th Betti number (β_i), which counts the number of independent i -dimensional holes (e.g., β_0 counts connected components, β_1 counts loops/tunnels). Persistent Homology tracks how these homology groups evolve (or persist) across the filtration. Features born at one scale and dying at a later scale are considered persistent and are deemed more likely to represent true topological structure rather than noise.

One-Parameter Persistence and Barcodes/Diagrams In the classical one-parameter setting (indexed by a single scale value, $t \in \mathbb{R}$), there exists a canonical and complete discrete invariant for PH: the persistence barcode.

Barcode: A multiset of intervals $[b, d]$, where b is the filtration value (scale) at which a topological feature is born (appears) and d is the value at which it dies (is filled or disappears).

Persistence Diagram (PD): An equivalent representation where each interval $[b, d]$ is plotted as a point (b, d) in the extended plane. The persistence (lifetime) of the feature is $d - b$, which is the vertical distance from the point to the diagonal $\Delta = (x, x)$, where noise typically lies.

• Persistence Diagram

A persistence diagram in degree k , denoted $Dg_k(f)$ or simply D_k , associated with a filtered simplicial (or cell) complex $K_\bullet = \{K_t\}_{t \in \mathbb{R}}$ induced by a real-valued function $f : X \rightarrow \mathbb{R}$ (typically assumed 1-Lipschitz or at least tame), is a locally finite multiset of points in the extended half-plane

$$\overline{\mathbb{R}^2}_+ = \{(b, d) \in \mathbb{R}^2 \mid b \leq d\} \cup \{(\cdot, \infty)\}$$

augmented by the diagonal $\Delta = \{(t, t) \mid t \in \mathbb{R}\}$ with infinite multiplicity. Each off-diagonal point (b_i, d_i) with $b_i < d_i < \infty$ records the birth and death parameters of a k -dimensional homology class $\alpha \in H_k(K_{b_i}; \mathbb{Z}_2)$ (or integer coefficients) that is created at filtration value $t = b_i$ via an inclusion-induced map

$$H_k(K_{b_i-}) \rightarrow H_k(K_{b_i})$$

having non-trivial image on α , and subsequently becomes trivial (dies) at $t = d_i$ through the map

$$H_k(K_{d_i-}) \rightarrow H_k(K_{d_i}).$$

Formally, for tame filtrations - those in which each K_t is a finite simplicial complex and only finitely many homology classes are born and die in any bounded interval - the persistence module

$$\mathbb{V}_k(t) := H_k(K_t)$$

is pointwise finite-dimensional (p.f.d.) and decomposes uniquely (up to reordering) as a direct sum of interval modules:

$$\mathbb{V}_k \cong \bigoplus_i \mathbb{I}_{[b_i, d_i]} \oplus \bigoplus_j \mathbb{I}_{[c_j, \infty)}$$

The finite-lifetime intervals $[b_i, d_i]$ correspond exactly to the off-diagonal points of D_k . The structure theorem for p.f.d. persistence modules (Crawley-Boevey, Gabriel, Zomorodian-Carlsson) guarantees that D_k is a complete invariant of $\mathbb{V}_k$. Stability is the cornerstone: if f and g are tame and $\|f - g\|_\infty \leq \epsilon$, then the bottleneck distance satisfies

$$d_B(D_k(f), D_k(g)) \leq \epsilon,$$

and more generally the p -Wasserstein distance obeys $W_p(D_k(f), D_k(g)) \leq \|f - g\|_\infty^{1/p}$ for $p \in [1, \infty]$. For 1-Lipschitz functions on compact domains, or sublevel-set filtrations of continuous functions on triangulable spaces, these conditions are satisfied, making persistence diagrams the central object of applied and computational topology.

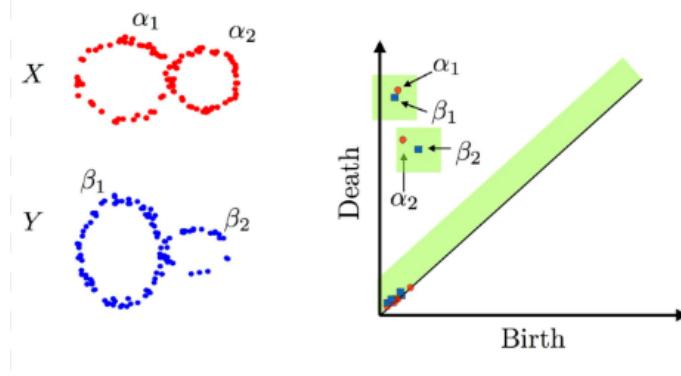


Figure 16. example of H0, H1 homologies (segments/voids) persistence diagram

• Persistence Barcode

A persistence barcode $\mathcal{B}_k(f)$ is the collection of half-open intervals $[b_i, d_i)$ (sometimes closed $[b_i, d_i]$) representing the support of indecomposable summands of the k -th persistence module $\mathbb{V}_k(t) = H_k(K_t; \mathbb{k})$ arising from a filtered complex $K_\bullet$. By the isometry theorem between barcode and diagram representations, there is a canonical bijection

$$(b, d) \in D_k \longleftrightarrow [b, d) \in \mathcal{B}_k.$$

The barcode viewpoint emphasizes the decomposition theorem: any pointwise finite-dimensional persistence module over $\mathbb{R}$ (i.e a functor $\mathbb{R} \rightarrow \mathbf{Vect}_{\mathbb{k}}$ with finite-dimensional values and tame transitions) is uniquely isomorphic to a direct sum of interval modules $\mathbb{1}_{[a,b)}$. Tameness is guaranteed when the filtration is induced by a monotone, Lipschitz, or piecewise-linear function on a finite simplicial complex, or by sublevel sets of a continuous function on a compact triangulable space. Stability inherits directly from diagrams: if $\|f - g\|_\infty \leq \varepsilon$, the Hausdorff distance between barcode collections satisfies

$$d_H(\mathcal{B}_k(f), \mathcal{B}_k(g)) \leq \varepsilon$$

, and the interleaving distance between modules is bounded by ε . Barcodes are particularly intuitive for visualization and for defining derived statistics such as total persistence $\sum_i (d_i - b_i)^p$, bottleneck length of the longest bar, or entropy of the length distribution. They are also the foundation of template-based methods (e.g., Template Functions, Persistence Codebooks) and are used in topological optimization via subgradient flows on barcode endpoints.

• Persistence Landscape

The persistence landscape, introduced by Bubenik (2015), is a functional representation that embeds a persistence diagram D into a separable Banach space of sequences of functions. For each point

$$(b_i, d_i) \in D$$

, define the piecewise-linear tent function

$$f_{(b_i, d_i)}(t) = \max\{0, \min(t - b_i, d_i - t)\}$$

Order these functions by pointwise value and define the k -th landscape function via

$$\lambda_k(t) = k - \sup\{f_i(t) \mid i\},$$

i.e., the k -th largest value at t (with $\lambda_k \equiv 0$ if fewer than k tents are non-zero). The full landscape is the sequence $\Lambda = (\lambda_k)_{k \geq 1} \in \ell^p(L^\infty(\mathbb{R}))$. Crucially, for tame filtrations (Lipschitz or PL functions on compact polyhedra), the map $D \mapsto \Lambda$ is 1-Lipschitz with respect to bottleneck distance:

$$\|\Lambda_1 - \Lambda_2\|_\infty \leq d_B(D_1, D_2).$$

Moreover, the inverse map is continuous, making landscapes a stable and invertible vectorization. The L^p -norm $\|\Lambda\|_p = (\sum_k \int |\lambda_k(t)|^p dt)^{1/p}$ induces a metric on the space of diagrams equivalent to the p -Wasserstein metric up to constants. Because landscapes live in a Banach space, classical statistical tools – means, variances, PCA, functional PCA, kernel methods apply directly without further discretization. They are especially powerful in hypothesis testing (permutation tests on landscape norms) and in regression settings where topological features are predictors.

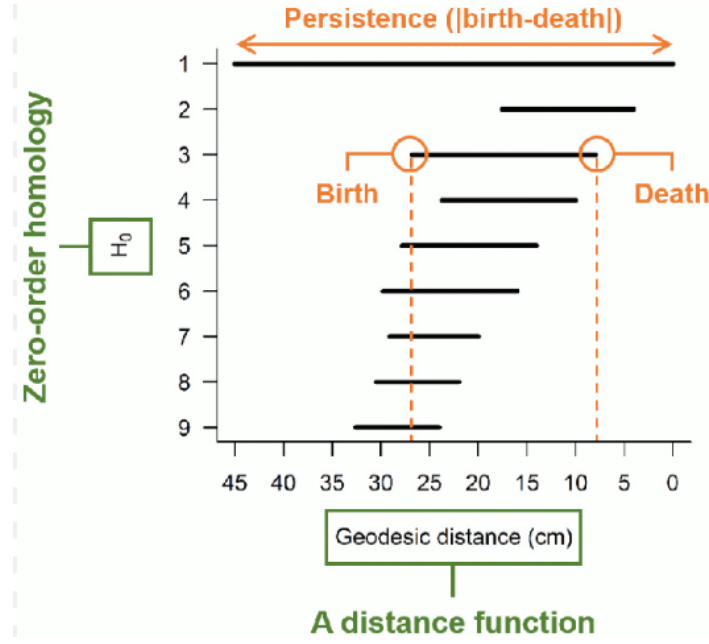


Figure 17. Barcode of above persistence diagram showcasing birth and death times

• Persistence Landscape

The persistence landscape, introduced by Bubenik (2015), is a functional representation that embeds a persistence diagram D into a separable Banach space of sequences of functions. For each point $(b_i, d_i) \in D$, define the piecewise-linear tent function

$$f_{(b_i, d_i)}(t) = \max\{0, \min(t - b_i, d_i - t)\}.$$

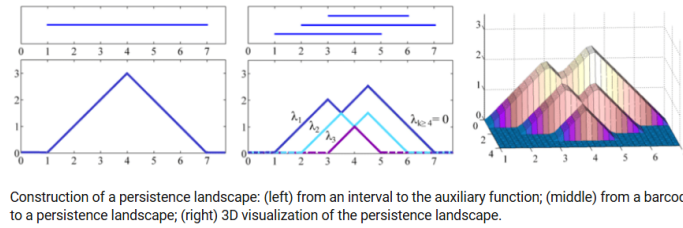
Order these functions by pointwise value and define the k -th landscape function via

$$\lambda_k(t) = k - \sup\{f_i(t) \mid i\},$$

i.e., the k -th largest value at t (with $\lambda_k \equiv 0$ if fewer than k tents are non-zero). The full landscape is the sequence $\Lambda = (\lambda_k)_{k \geq 1} \in \ell^p(L^\infty(\mathbb{R}))$. Crucially, for tame filtrations (Lipschitz or PL functions on compact polyhedra), the map $D \mapsto \Lambda$ is 1-Lipschitz with respect to bottleneck distance:

$$\|\Lambda_1 - \Lambda_2\|_\infty \leq d_B(D_1, D_2).$$

Moreover, the inverse map is continuous, making landscapes a stable and invertible vectorization. The L^p -norm $\|\Lambda\|_p = (\sum_k \int |\lambda_k(t)|^p dt)^{1/p}$ induces a metric on the space of diagrams equivalent to the p -Wasserstein metric up to constants. Because landscapes live in a Banach space, classical statistical tools— means, variances, PCA, functional PCA, kernel methods— apply directly without further discretization. They are especially powerful in hypothesis testing (permutation tests on landscape norms) and in regression settings where topological features are predictors.



Construction of a persistence landscape: (left) from an interval to the auxiliary function; (middle) from a barcode to a persistence landscape; (right) 3D visualization of the persistence landscape.

Figure 18. Persistence Landscape example

• Persistence Image

Persistence images (Adams et al., 2017) provide a differentiable, finite-dimensional, and stable vectorization of diagrams. Given D_k , transform each point (b_i, d_i) to birth-persistence coordinates $(b_i, \pi_i) = (b_i, d_i - b_i)$. Choose a resolution grid $\mathcal{G} \subset \mathbb{R}^2$ and a differentiable weighting function $w : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$ (typically $w(\pi) = \pi^2$ or $w(\pi) = \arctan(C\pi^p)$ to emphasize long-lived features). The persistence surface is the weighted sum of 2D Gaussians (or other kernels):

$$\rho(x, y) = \sum_i w(\pi_i) \cdot \phi_\sigma((x, y) - (b_i, \pi_i))$$

where ϕ_σ has variance σ^2 . Discretizing ρ over $\mathcal{G}$ yields a vector $\mathbf{v} \in \mathbb{R}^{mn}$. For tame filtrations of 1-Lipschitz functions, the map $f \mapsto \mathbf{v}$ is Lipschitz with respect to the W_1 -metric on diagrams and the Euclidean metric on images, with explicit constants depending only on σ and the weighting function's Lipschitz constant. The Gaussian kernel ensures differentiability almost everywhere, enabling backpropagation through topological features in deep learning architectures (e.g., TopoNet, PersLay). Persistence images are currently the most widely used vectorization in machine learning applications of TDA because they combine stability, differentiability, and computational efficiency.

- (Heat Kernel / Image) The persistence heat kernel signature was introduced by Reininghaus et al. (2015) and later generalized in PersLay (Carriere et al, 2020) provides a smooth, differentiable, and stable embedding of persistence diagrams into a reproducing kernel Hilbert space (RKHS). Given a

persistence diagram $D_k = \{(b_i, d_i)\}_{i \in I}$ in degree k , and a positive scale parameter $t > 0$, the heat kernel embedding is the function

$$K_t(D_k)(x, y) := \sum_{(b_i, d_i) \in D_k} \exp\left(-\frac{\|(x, y) - (b_i, d_i)\|^2}{8t}\right), \quad (x, y) \in \mathbb{R}^2$$

This is the unique solution to the heat equation $\partial_t u = \Delta u$ with initial condition $u(0, \cdot, \cdot) = \sum_i \delta_{(b_i, d_i)}$ on $\mathbb{R}^2$. Because the Gaussian kernel is positive definite, $K_t(D_k)$ defines a point in an RKHS $\mathcal{H}_t$ associated to the Gaussian kernel of variance $4t$. The map $D_k \mapsto K_t(D_k)$ is 1-Lipschitz from the space of tame diagrams equipped with the 2-Wasserstein metric W_2 into $\mathcal{H}_t$ equipped with its native norm:

$$\|K_t(D_k) - K_t(D'_k)\|_{\mathcal{H}_t} \leq W_2(D_k, D'_k).$$

This follows from the fact that the Gaussian kernel has Lipschitz constant $1/(2\sqrt{2t})$ in L^2 under optimal transport plans. For filtrations induced by 1-Lipschitz functions f, g on a compact metric space (X, d) , the sublevel-set persistence diagrams satisfy $W_2(D_k(f), D_k(g)) \leq \|f - g\|_\infty$, so the embedding is stable under uniform perturbation of the filter function. Differentiability is immediate: the gradient $\nabla_\theta K_t(D_k(f_\theta))$ with respect to parameters θ of f can be computed via the chain rule through the persistent homology layer, enabling end-to-end training in deep topological models. The multi-scale signature $t \mapsto K_t(D_k)$ (often discretized over a logarithmic grid) yields a stable feature map used in support vector machines, kernel PCA, and neural architectures such as PersLay, which learns optimal linear combinations of such embeddings across scales and dimensions.

- (Betti Numbers / Curves) Given a filtered topological space $\{X_t\}_{t \in \mathbb{R}}$ (typically sublevel sets $X_t = f^{-1}(-\infty, t]$ of a continuous tame function $f : X \rightarrow \mathbb{R}$ on a compact triangulable space), the k -th Betti curve is the integer-valued function

$$\beta_k(t) := \dim H_k(X_t; \mathbb{Z}_2) = \text{rank}(H_k(X_{-\infty}) \rightarrow H_k(X_t))$$

It is non-decreasing, right-continuous, piecewise constant, with jumps precisely at critical values of f where new k -dimensional homology classes are born. For tame filtrations (finite number of homological critical values), $\beta_k(t)$ is bounded and eventually constant. The vector-valued Betti curve $\beta(t) = (\beta_0(t), \beta_1(t), \beta_2(t), \dots)$ is a stable topological signature: if f and g are continuous and $\|f - g\|_\infty \leq \varepsilon$, then Cohen-Steiner, Edelsbrunner, and Harer (2009) prove that

$$\sup_t |\beta_k^f(t) - \beta_k^g(t)| \leq 1$$

for all k and t , with equality possible only at birth events. This L^∞ stability is weaker than bottleneck stability but computationally cheaper: Betti curves require only tracking ranks of boundary matrices, not full persistence pairings. They are widely used in topological data analysis for scalar fields (e.g., density functions, distance-to-measure), in sensor networks (coverage analysis), and as lightweight features in machine learning pipelines where full persistence diagrams are too expensive.

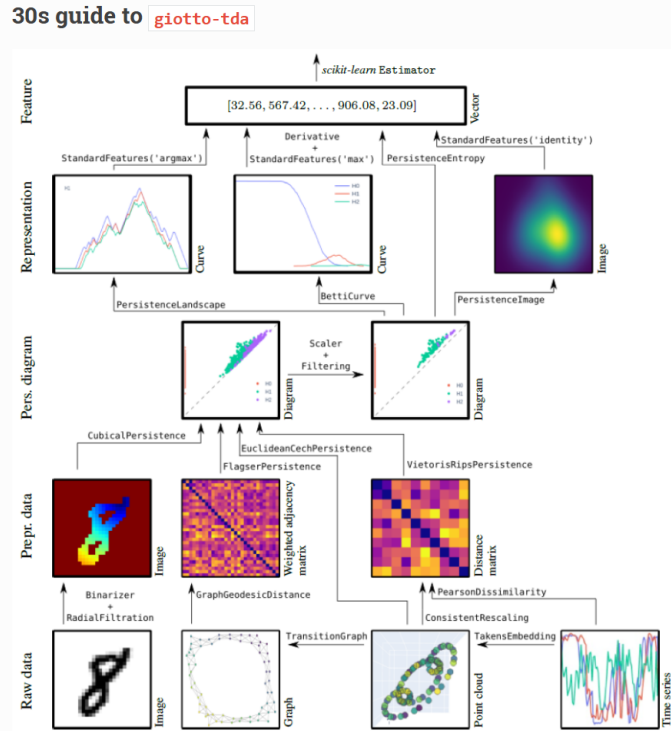


Figure 19. Betti numbers/curves as TDA features

- (Bottleneck Distance) The bottleneck distance, introduced by Cohen-Steiner, Edelsbrunner, and Harer (2007), is the canonical L^∞ -optimal matching distance between persistence diagrams. Let D_1, D_2 be two diagrams in any degree, augmented with the diagonal Δ of infinite multiplicity. A partial matching $\gamma : D_1 \rightarrow D_2$ allows unpaired points to be matched to their projection onto Δ at cost $|d - b|/2$. The bottleneck distance is

$$d_B(D_1, D_2) := \inf_{\gamma} \sup_{p \in D_1} \|p - \gamma(p)\|_\infty,$$

where $\|\cdot\|_\infty$ is the Chebyshev distance on $\mathbb{R}^2$. The infimum is achieved (by the Hungarian algorithm in polynomial time for finite diagrams). For sublevel-set filtrations of ε -close 1-Lipschitz functions f, g on a common compact triangulable space, the stability theorem states

$$d_B(\text{Dg}_k(f), \text{Dg}_k(g)) \leq \|f - g\|_\infty \leq \varepsilon.$$

This is the strongest known stability result for persistent homology: no weaker metric (e.g., Hausdorff) achieves better constants. The bottleneck distance metrizes the space of tame persistence diagrams and induces the Gromov–Hausdorff topology on the space of compact metric spaces via the persistence distortion distance.

- **p -Wasserstein Distance**

The p -Wasserstein distance ($1 \leq p < \infty$) between persistence diagrams D_1, D_2 is defined via optimal mass transportation with cost $\|p - q\|_2^p$:

$$W_p(D_1, D_2)^p := \inf_{\gamma: D_1 \rightarrow D_2} \sum_{p \in D_1} \|p - \gamma(p)\|_2^p,$$

where γ is a partial bijection (unmatched points pay the cost to the diagonal). When $p \rightarrow \infty$, W_p converges to the bottleneck distance. For tame filtrations of 1-Lipschitz functions f, g on compact spaces, the stability theorem of Cohen-Steiner et al. gives

$$W_p(D_k(f), D_k(g)) \leq \|f - g\|_\infty^{1-\frac{1}{p}} \cdot C_p,$$

with explicit constants C_p depending only on p and diagram cardinality. The $p = 2$ case (Earth Mover's Distance) is particularly important: it is Hilbert-metrized by the persistence Fisher information metric and appears naturally in gradient flows on diagram space. W_2 is stable, differentiable almost everywhere, and widely used in topological optimization and statistical inference.

- (Sliced Wasserstein Distance) The sliced Wasserstein distance (Rabin et al., 2011; Carriere et al., 2020) approximates W_p via random projections. For unit vector $\theta \in S^1$, project a diagram D radially:

$$\pi_\theta(b, d) := b \cdot \theta_1 + d \cdot \theta_2.$$

Then

$$SW_p(D_1, D_2)^p := \mathbb{E}_{\theta \sim \mathcal{U}(S^1)} [W_p(\pi_\theta(D_1), \pi_\theta(D_2))]^p.$$

The expectation is approximated by Monte-Carlo over N directions. For tame diagrams, SW_p is bounded by W_p up to constants depending only on p and dimension 2. Crucially, SW_p is differentiable almost everywhere and $O(Nn \log n)$ computable, making it the metric of choice in deep learning with persistence (e.g., TopoLoss, PersLay). Stability inherits from W_p : if $\|f - g\|_\infty \leq \epsilon$, then $SW_p(D(f), D(g)) = O(\epsilon^{1-1/p})$.

- **Multiparameter Persistence and the Rank Invariant**

Multiparameter persistence generalizes classical persistent homology to filtrations indexed by $\mathbb{R}^n$ or $\mathbb{Z}^n$ with the product order $\leq$. Given a bifiltered (or n -filtered) simplicial complex $K_t = K_{t_1, \dots, t_n}$ with inclusions $K_s \hookrightarrow K_t$ whenever $s \leq t$, the associated n -parameter persistence module is the functor

$$V : (\mathbb{R}^n, \leq) \rightarrow \mathbf{Vect}_k, \quad t \mapsto H_k(K_t; k).$$

Under the tameness assumption: only finitely many homology classes born/die along any line, and finite-dimensional images, V is pointwise finite-dimensional (p.f.d.). Unlike the 1-parameter case, no complete discrete invariant analogous to barcodes exists (Carlsson–Zomorodian, 2009). The fundamental result is that the only complete invariant is the rank invariant (or bigraded Betti numbers):

$$\rho_V(\mathbf{a}, \mathbf{b}) := \text{rank}(V(\mathbf{a}) \rightarrow V(\mathbf{b})), \quad \mathbf{a} \leq \mathbf{b}.$$

This defines a function $\rho : \{(\mathbf{a}, \mathbf{b}) \mid \mathbf{a} \leq \mathbf{b}\} \rightarrow \mathbb{N}$ that fully characterizes V up to isomorphism. The signed barcode decomposition fails in dimension $n \geq 2$ due to non-interval indecomposables (Lesnick, 2015). The associated graded module over the polynomial ring $k[x_1, \dots, x_n]$ is finitely generated, and its Hilbert series or minimal free resolution encode topological complexity. Stability holds in the interleaving distance: if two bifiltrations are ϵ -interleaved (there exist natural transformations shifting by ϵ in each direction composing to inclusions shifted by 2ϵ), then their rank invariants differ by at most $O(\epsilon)$. The Fibonacci module and non-rectangular indecomposables show that multiparameter modules are wild. Applications include multifiltered shape analysis (e.g., density + height), symmetry-aware TDA, and topological inference under multiple nuisance parameters. Recent advances include the persistent homology of real-valued functions on posets, the theory of Mobius inversion for rank invariants (Botnan–Lesnick, 2021), and the development of the RHom complex for computing derived limits in multiparameter settings.

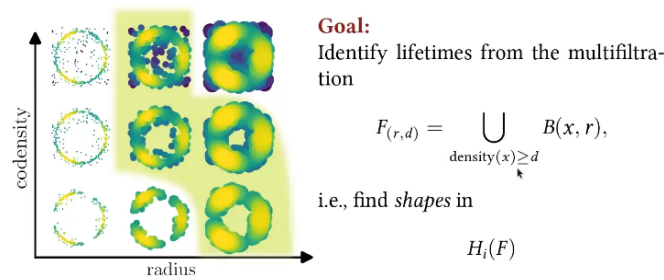


Figure 20. example of 2 parameter persistence

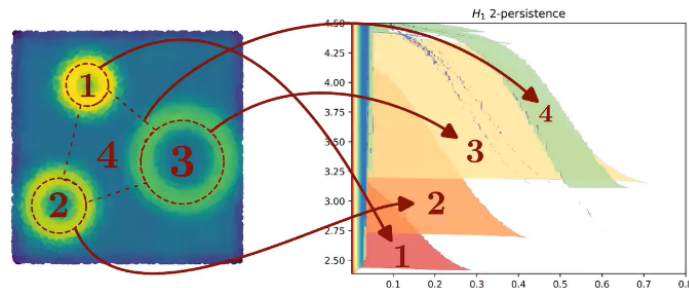


Figure 21. example of the mapping of each region of the data showing overlap with colours

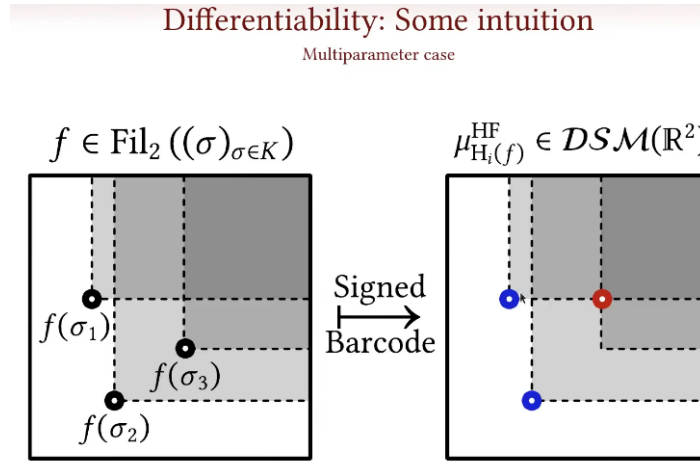


Figure 22. Recent contribution of assigning positive/negative measures to distinguish overlapping surfaces

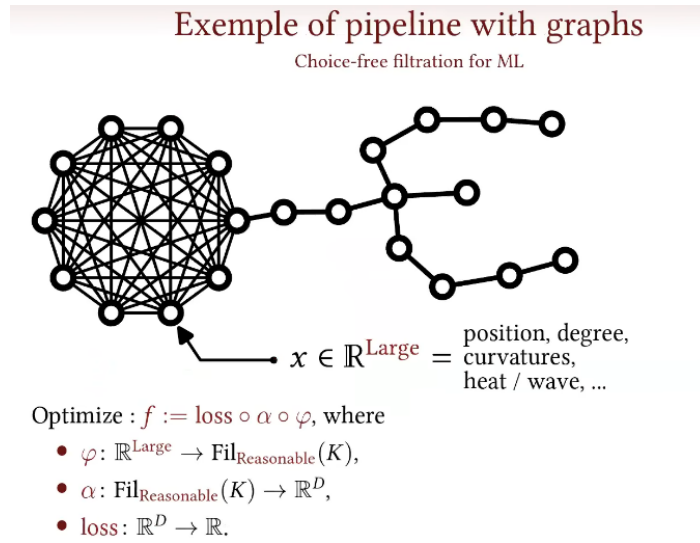


Figure 23. the multipers package is also capable of finding the optimal filtration function thanks to Brenier's rearrangement

• Mapper (TDA Clustering Graph / Reeb Graph Approximation)

The Mapper algorithm (Singh–Memoli–Carlsson, 2007) constructs a combinatorial summary of high-dimensional data $X \subset \mathbb{R}^d$ using a filter function

$$f: X \rightarrow \mathbb{R}^m$$

(e.g., density, eccentricity, PCA coordinates). Cover

$$\text{range}(f)$$

by overlapping open sets $\{U_\alpha\}_{\alpha \in A}$, pull back to

$$f^{-1}(U_\alpha)$$

, and cluster each preimage (e.g., single linkage). Nodes of the Mapper graph $M(f, \mathcal{U})$ are clusters $C_{\alpha,i}$, and edges connect clusters with non-empty intersection in X . Formally,

$$M(f, \mathcal{U}) = \text{Nerve}(\{C_{\alpha,i}\}_{\alpha,i}).$$

Under the nerve theorem, if the cover is sufficiently fine and clusters respect connected components,

$$M(f, \mathcal{U})$$

is homotopy equivalent to the Reeb space of f . For 1-dimensional filters ($m = 1$), Mapper approximates the Reeb graph; for higher m , the Reeb space (a stratification by connected components of level sets). Stability was proven by Carriere–Oudot (2018): if f, g are ε -close in sup norm and the cover has Lebesgue number

$$\delta > 2\varepsilon$$

, then the Mapper graphs are

$$(\varepsilon, \delta)$$

–interleaved, implying bounded Gromov–Hausdorff distance after geometric realization. Mapper is widely used in biological data analysis (protein interaction networks, cancer genomics), neuroscience (functional brain networks), and sensor coverage. Variants include Multiscale Mapper, Graph Induced Complex, and Joint Mapper for multiple functions.

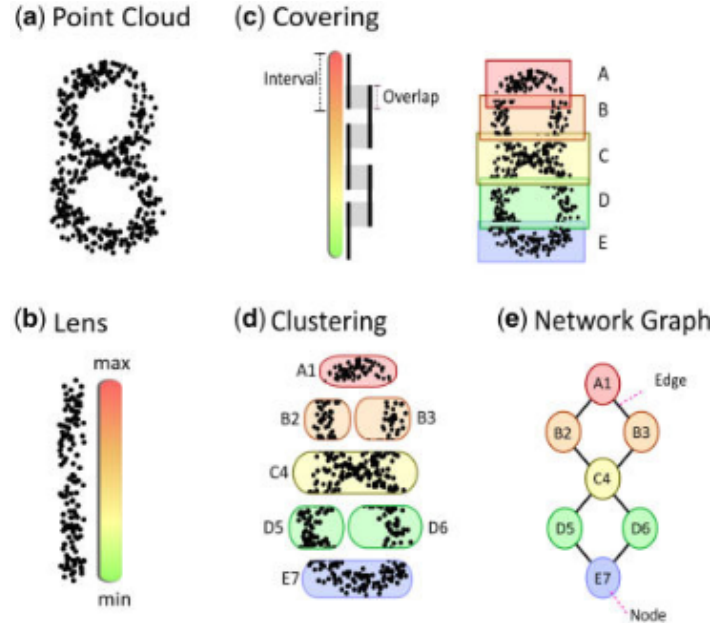


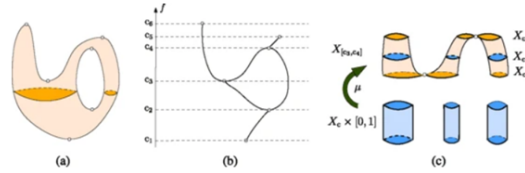
Figure 24. Steps to the TDA algorithm for point cloud simplification and TDA

• Reeb Graph

Given a compact topological space X and a continuous scalar function $f : X \rightarrow \mathbb{R}$, the Reeb graph R_f is the quotient space

$$R_f := X / \sim_f, \quad x \sim_f y \iff f(x) = f(y) \text{ and } x, y \text{ lie in the same connected component of } f^{-1}(f(x)).$$

Equipped with the quotient topology and induced function $\bar{f} : R_f \rightarrow \mathbb{R}$, it forms a 1-dimensional stratified space with nodes at critical points (minima, maxima, saddles). The Reeb graph is functorial: continuous maps commuting with f induce maps on R_f . For tame Morse functions on compact manifolds, R_f has finitely many nodes and edges. Stability in the functional distortion distance was proven by Bauer-Munch (2015): if $\|f - g\|_\infty \leq \varepsilon$ and the functions are δ -close in C^0 on level sets, then $d_{GH}(R_f, R_g) \leq O(\varepsilon + \delta)$. The mapping theorem (Edelsbrunner-Harer-Zomorodian, 2003) shows that Reeb graphs classify scalar fields up to contour reconnection. Applications include shape matching, topological simplification, and feature extraction in scientific visualization.



(a) X is a solid torus and its Reeb graph w.r.t. the height function f is shown in (b). (c) f is a level-set-tame function w.r.t. discrete values $\{c_1, \dots, c_6\}$. There is a continuous map $\mu : X_c \times [0, 1] \rightarrow X_{[c_3, c_4]}$ whose restriction to the open set $X_c \times (0, 1)$ is a homeomorphism. In the top row, there are three disjoint interval-components in $X_{(c_3, c_4)}$ whose closures may intersect in level sets X_{c_3} and X_{c_4} .

Figure 25. Example of a reeb graph of a data cloud

- **Contour Trees** When M is simply connected, the Reeb graph is a tree called the contour tree. Nodes correspond to critical points of f , edges to monotone paths between them.

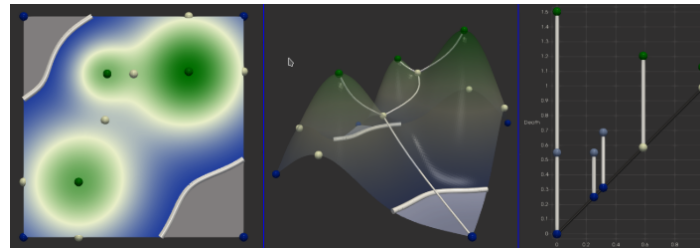


Figure 26. Contour Trees

• Merge Tree (and Join/Split Trees)

The merge tree (or contour tree when the domain is simply connected) tracks the evolution of connected components of sublevel sets $X_{\leq t} = f^{-1}(-\infty, t]$. Nodes correspond to critical points of f : leaves are local minima, internal nodes are saddles where components merge. Formally, the merge tree is a rooted tree with height function induced by f . For PL functions on triangulable spaces, it is computable in $O(n \log n)$ via union-find with path compression.

Persistence-based simplification removes low-persistence merges. Stability was established by Morozov (2007): the edit distance between merge trees of ε -close functions is $O(\varepsilon)$. Merge trees are dual to contour trees and fundamental in volumetric data analysis, topological data reduction, and uncertainty visualization.

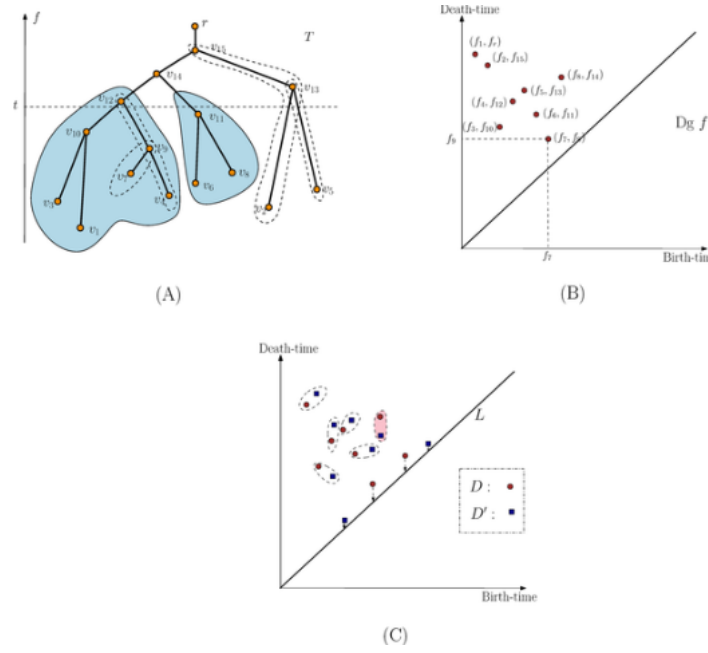


Figure 27. Using data sketching to cluster Contour trees together and simplify sufficiently similar data

• Morse–Smale Complex

Given a compact smooth manifold M and a Morse function $f : M \rightarrow \mathbb{R}$ satisfying the Morse–Smale transversality condition (stable and unstable manifolds intersect transversely), the Morse–Smale complex is the CW-complex obtained by intersecting ascending manifolds of minima/maxima with descending manifolds of saddles. Cells are gradient flow lines connecting critical points. The 1-skeleton is the union of Reeb graph and its dual (split tree). Edelsbrunner–Harer–Zomorodian (2003) gave a discrete algorithm using simulated gradient flow on PL manifolds. Persistence simplification removes low-persistence critical pairs while preserving homotopy type. Stability follows from the stability of critical points under C^2 perturbations (Edelsbrunner–Harer, 2010). The Morse–Smale complex is the gold standard for topological feature segmentation in scalar fields (e.g., combustion simulation, climate modeling).

• Topological Regularizers and Differentiable Topology

Topological regularization incorporates persistent homology into machine learning objectives to enforce desired global structure. Given a parameterized function $f_\theta : X \rightarrow \mathbb{R}$ (e.g., a neural network score function), compute its sublevel-set persistence diagram $D_k(f_\theta)$. A topological regularizer is a differentiable penalty of the form

$$\mathcal{L}_{\text{topo}}(\theta) = \sum_{(b_i, d_i) \in D_k(f_\theta)} \phi(d_i - b_i),$$

where

$$\phi : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}$$

is increasing and subgradients exist almost everywhere (e.g.,

$$\phi(p) = p$$

, p^2 , or a soft hinge). To enable backpropagation, one uses differentiable vectorizations: persistence images (Adams et al., 2017), landscapes (Bubenik, 2015), or the PersLay framework (Carriere et al., 2020), which learns optimal linear combinations of kernel embeddings. Stability ensures robustness: if

$$\|f_\theta - f_{\theta'}\|_\infty \leq \varepsilon$$

, then

$$W_2(D_k(f_\theta), D_k(f_{\theta'})) \leq \varepsilon$$

, so the regularizer is 1-Lipschitz in the parameter space under mild assumptions on the architecture (Hofer et al., 2019). Applications include enforcing connectivity in segmentation (Hu et al., 2019), removing spurious cycles in generative models (Moor et al., 2020), and learning manifold-structured representations. Recent work combines topological losses with optimal transport (Gabriel et al., 2023) and defines subgradients directly on barcode endpoints via the Eilenberg–MacLane correspondence.

• Local Cohomology and Relative Homology in TDA

Local cohomology measures topological obstructions near a subset $Z \subset X$. For a closed subset Z and sheaf $\mathcal{F}$ on a triangulable space X , the i -th local cohomology group with support in Z is

$$H_Z^i(X; \mathcal{F}) := \lim_{U \supset Z} H^i(U, U \setminus Z; \mathcal{F}).$$

In applied TDA, one uses Čech or derived local cohomology on point clouds with a distinguished subset (e.g., boundary points). For a pair (X, A) , relative persistent homology

$$H_k(X_t, A_t)$$

captures features that “touch” A and die when merged into it. Local cohomology appears naturally in the persistence Mayer–Vietoris sequence and in the study of stratified spaces. Bendich et al. (2012) used local homology to detect boundaries and singularities in sampled manifolds. Stability follows from the algebraic stability theorem applied to the long exact sequence of the pair: perturbations of size ε induce at most ε -interleavings in local cohomology modules. Recent generalizations include persistent local cohomology for multifiltered spaces (Scoccola, 2022).

- **Vietoris–Rips Filtration**

Given a finite point cloud

$$X \subset \mathbb{R}^d$$

with metric

$$d$$

, the Vietoris–Rips complex at scale $r > 0$ is the abstract simplicial complex

$$VR_r(X) := \{\sigma \subset X \mid \text{diam}(\sigma) \leq 2r\}.$$

The associated filtration $r \mapsto VR_r(X)$ is 1–Lipschitz in the Gromov–Hausdorff sense: if $d_{GH}(X, Y) \leq \varepsilon$, then

$$VR_r(X)$$

and

$$VR_{r+2\varepsilon}(Y)$$

have a common refinement. The reach theorem implies that $VR_r(X)$ and the reach complex at scale r are homotopy equivalent when r is below half the reach of the underlying sampled manifold (Chazal et al., 2017). Persistent homology of the Vietoris–Rips filtration is the most widely used TDA pipeline due to its metric simplicity and $O(n^3)$ matrix reduction complexity (via Ripser, Bauer 2021). Stability follows from the stability of 1–Lipschitz functions: distance–to–point functions are 1–Lipschitz, yielding

$$d_B(D_k(X), D_k(Y)) \leq 2d_{GH}(X, Y)$$

- **Cech Complex**

The Cech complex at scale r is

$$\check{C}_r(X) := \left\{ \sigma \subset X \mid \bigcap_{x \in \sigma} B(x, r) \neq \emptyset \right\}.$$

It is the exact nerve of the r –ball cover, hence homotopy equivalent to the r –offset X^r by the nerve theorem. For 1–Lipschitz functions (e.g., distance–to–measure), the Cech filtration is stable under Hausdorff perturbation: $d_H(X, Y) \leq \varepsilon$ implies $d_B(D_k(\check{C}(X)), D_k(\check{C}(Y))) \leq \varepsilon$. Computationally expensive (exponential in dimension), it is used theoretically and in low–dimensional settings (e.g., GUDHI’s Cech persistence).

- **Alpha Complex and Delaunay Filtration**

The alpha complex $\alpha_r(X)$ of a point cloud $X \subset \mathbb{R}^d$ is the subcomplex of the Delaunay triangulation whose simplices have circumradius

$$\leq \alpha$$

and whose dual Voronoi cells intersect non–trivially. Equivalently, it is the nerve of balls of radius $\sqrt{\alpha}$ centered at points, restricted to Delaunay–consistent intersections. Edelsbrunner (1995) proved that $\alpha_r(X)$ is homotopy equivalent to the union of balls of radius $\sqrt{\alpha}$, hence to the offset (sublevel set) of the distance–to– X function. The alpha filtration is intrinsically geometric: it avoids the exponential size of Cech complexes while preserving homotopy type up to the reach of the sampled manifold. Stability is stronger than Vietoris–Rips: perturbations of points by ε induce

$$O(\varepsilon)$$

–changes in alpha diagrams. GUDHI (Maria et al., 2014) and Dionysus implement efficient alpha complex persistence.

- **Cubical Complexes**

A cubical complex is built by gluing elementary cubes

$$I = [i_0, i_0 + 1] \times \cdots \times [i_d, i_d + 1]$$

, $i \in \mathbb{Z}^d$, along faces. For grayscale image data

$$f : \mathbb{Z}^d \rightarrow \mathbb{R}$$

, the lower–star filtration includes a cube when all its vertices satisfy

$$f(v) \leq t$$

. Cubical homology (Kaczynski–Mischaikow–Mrozek, 2004) exploits bitmap structure: boundary maps are ± 1 on adjacent voxels, yielding sparse matrices and

$$O(n)$$

persistence computation via incremental cohomology. For Lipschitz functions on regular grids, cubical persistence is stable under L^∞ perturbation. Applications dominate digital image analysis, porous media, and neuroscience connectomics. Software: Perseus, Cubical Ripser.

TDA Feature	multisets	Graphs	graphs-like	Persity	TDK	Regner	Persim	Regner Mapper	Topological layer	Search Topological	Differentiable?
Persistence Diagram	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	Partial
Barcode	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	Partial
Persistence Landscape	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No
Persistence Image	✗	✓	✓	✓	✗	✗	✓	✗	✓	✓	No
Heat Kernel/Image	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	No
Betti Numbers/Curves	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No
Bottleneck Distance	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No
Wasserstein Distance	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No
Sliced Wasserstein Distance	✓	✓	✗	✓	✗	✗	✓	✗	✓	✓	No
Multiparameter Persistence	✓	Partial	✗	✗	✗	✗	✗	✗	✗	✗	No
Mapper Algorithm/Graph	✗	✓	✓	✓	✗	✗	✗	✓	✗	✗	No
Reed Graph	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗	No
Contour Trees	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗	No
Merge Trees	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗	No
Morse-Smale Complex	✗	✓	✗	✗	✓	✗	✗	✗	✗	✗	No
Topological Regularizer	✓	✗	✗	✗	✗	✗	✗	✗	✓	✓	No
Local Cohomology	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	No
Vietoris-Rips Complex	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No
Alpha Complex	✗	✓	✓	✓	✗	✗	✗	✗	✗	✗	No
Cubical Complex	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓	No

✓ Fully Supported Partial Partially Supported ✗ Not Supported Differentiable? Indicates gradient descent compatibility for ML integration

Figure 28. Available tools and what features they support

Algorithm 2: Mapper algorithm

Input: D with $|D| = m$, filter function $f : D \rightarrow \mathbb{R}^d$, finite cover $\mathcal{U} = \{U_i\}_{i \in I}$ of $\text{Im}(f) \subseteq \mathbb{R}^d$, clustering algorithm $\mathcal{C}$.
Output: Simplicial complex S_D .

```

1  $S_D \leftarrow \emptyset$ ;
2  $D_i \leftarrow f^{-1}(U_i)$  for all  $i \in I$ ;
3 for all  $i \in I$  do
4    $\{C_i^1, \dots, C_i^{k_i}\} \leftarrow \mathcal{C}(D_i)$  // Apply the clustering algorithm to  $D_i$ : the output are the clusters
5    $S_D \leftarrow S_D \cup \{C_i^1, \dots, C_i^{k_i}\}$  // Add the clusters found as vertices
6 for all  $\{C_1, \dots, C_t\} \in \mathcal{P} \setminus \mathcal{S} \cup_{i \in I} \{C_i^1, \dots, C_i^{k_i}\}$  // for all possible subsets of found clusters do
7   if  $\bigcap_{j=1}^t C_j \neq \emptyset$  then
8      $S_D \leftarrow S_D \cup \{C_1, \dots, C_t\}$  // We add the simplex  $\{C_1, \dots, C_t\}$ 
9 return  $S_D$ ;
```

The Mapper algorithm (introduced by Singh et al., 2007) is a tool for visualizing and extracting structural descriptions from high-dimensional datasets, typically outputting a graph or simplicial complex. Mapper uses four primary ingredients: 1. A dataset D . 2. A filter function (f): A scalar (usually $\mathbb{R}$ or $\mathbb{R}^2$ -valued) function defined on the dataset D that projects the high-dimensional data down to a manageable domain. 3. A finite, overlapping cover of the image of the filter function f . 4. A clustering algorithm (like DBSCAN) applied to the preimages of the cover elements. The result is a graph where nodes represent clusters of points that are close in the data space and map close to each other via the filter function, and edges connect nodes if their corresponding clusters overlap. Mapper is highly valuable for exploratory data analysis and visualization, offering interpretability, but the complex construction typically renders the graph structure non-differentiable for end-to-end optimization. 4. Transition to Multidimensional Persistence In many applications, data is complex and may be filtered along multiple non-linear dimensions simultaneously (e.g., scale and density), leading to a multifiltration. The critical transition occurs when $n > 1$ parameters index the filtration (e.g., a bifiltration parameterized by radius ϵ and curvature κ). Gunnar Carlsson and Afra Zomorodian demonstrated that, unlike the one-parameter case, no similar complete discrete invariant (like the barcode) exists for multiparameter persistence modules due to fundamental algebraic reasons. Instead, they proposed the rank invariant ($\rho_{X,i}$) as a robust discrete invariant for multifiltrations.

Rank Invariant: Defined as $\rho_{X,i}(u, v) = \text{rank}(H_i(X_u, k) \rightarrow H_i(X_v, k))$, this invariant measures the rank of the structure maps between homology groups indexed by parameters $u \leq v$.

While the rank invariant is mathematically equivalent to the persistence barcode in one dimension (where it is complete), it extends to higher dimensions as a useful, though incomplete, invariant for robust estimation of Betti numbers and detection of persistent features. To address the lack of a canonical discrete descriptor, modern research focuses on developing effective and often differentiable incomplete descriptors for multiparameter persistence, such as the multiparameter persistence landscape, multiparameter persistence image, and signed barcodes as measures.

Recommended Loss Function and Topological Descriptors To rival diffusion-based methods like D2NMG, which prioritize reconstruction accuracy (often L_2 based), my Transformer model must optimize for qualities conducive to fine-tuning and fast learning—qualities often missed by simple MSE on weights. Choosing the Optimal Loss Function Since my Transformer uses an autoencoder structure for weight regression, the loss should leverage known functional losses for hyper-representations combined with topologically robust metrics:

1. Layer-Wise Loss Normalization (LWLN): You should adopt the LWLN MSE loss. This loss prevents layers with small weight magnitudes from being disregarded, which is critical for generating functional models (as shown in work related to hyper-representations). $L_{MSE} = \frac{1}{MN} \sum_{i=1}^M \sum_{l=1}^L \frac{|\hat{w}_i^{(l)} - w_i^{(l)}|_2^2}{\sigma_i^2}$

2. Contrastive Loss (L_c): The original hyper-representation work combines L_{MSE} with a contrastive loss L_c to impose structure and regularity on the latent space. This practice should be retained: $L = \beta L_{MSE} + (1 - \beta) L_c$.

3. Topological Regularizer (Differentiable WD): To enforce manifold properties and bridge the gap to high-performing weights efficiently, incorporating the topological distance that showed success in my convergence experiments :the Wasserstein Distance (WD) is essential. The WD metric on persistence diagrams is the large-distance version of optimal transport distance. Since full WD is often non-differentiable, you should employ the Sliced Wasserstein Distance (SWD), which is known to be differentiable. $L_{Topological} = \alpha \cdot \text{SWD}(\text{PH}(\hat{W}), \text{PH}(W_{target}))$ Where $\hat{W}$ are the predicted weights and W_{target} are the ground truth/optimal weights.

4. Weight Importance Regularization (Fisher Information): Given my work's focus on continual learning and efficient representations, a term promoting finetunable weights is necessary. The Fisher Information Matrix (F) provides information about weight importance. Adopting an Elastic Weight Consolidation (EWC) style regularization term, where parameters are penalized based on their importance (F) to previous tasks, aligns perfectly with penalizing deviation from desired weight structures. $R = \sum_i \frac{1}{2} F_i (\hat{w}_i - w_i)^2$ Where w_i are the important weights/embeddings from the encoder stage (sub-partitions) and F_i is the corresponding diagonal Fisher information value. Recommended Composite Loss: A mixture that targets both precise reconstruction and topological/functional relevance: $L_{Total} = \beta L_{MSE} + (1 - \beta) L_c + \alpha L_{Topological} + R$

Summary of Differentiable Topological Descriptors :

The complexity of TDA features often requires turning the persistence diagram (PD) into a vectorization or using specific differentiable measures to integrate into gradient-based learning pipelines. The following table summarizes key descriptors and their suitability for Transformer training (where differentiability is crucial): Table 2: Topological Descriptors

Table 2. Topological Descriptors and Suitability for Transformer Training

TDA Feature	Description	Differentiable?	Notes/Context
Persistence Diagram/Barcode	Multiset of birth/death points.	Partial. Computation involves non-smooth operations. Requires implicit differentiation libraries (e.g., Torchph).	Core descriptor, often used as input for differentiable vectorizations.
Persistence Landscape	Continuous, piecewise linear function summarizing PD features.	Yes. Allows for averaging and is useful for statistical analysis.	One of the most computationally tractable differentiable descriptors.
Persistence Image (Heat Kernel)	Grid-based vectorization of PD using Gaussian kernels.	Yes. Provides a smooth, fixed-size vector output highly valued in ML.	Implemented for differentiability in libraries like Perslay and giotto-tda.
(Sliced) Wasserstein Distance	Metric based on optimal transport between PDs.	Yes (Sliced and kernelized versions). Full bottleneck distance is generally non-differentiable.	Crucial for manifold learning and metric preservation.
Signed Barcodes/Hilbert Decomposition	Discrete signed measure characterizing the Hilbert function of a filtration.	Yes (as measures).	These multiparameter descriptors are shown to be semilinearly determined on grids, allowing for explicit Clarke subdifferentiability.
Multiparameter Persistence	Persistence indexed by $n > 1$ parameters (e.g., density + scale).	Yes (via specific descriptors/libraries like multipers).	Generally challenging, but differentiable invariants are emerging.
Topological Regularizer	Topological distance used as a loss function component.	Yes.	Directly designed for integration into gradient descent pipelines.

Comparison of Simplicial Complexes The choice of simplicial complex construction determines the fidelity of the topological approximation and computational cost. For geometric data (like weights visualized as point clouds), Vietoris–Rips and related complexes are common. Table 3: Comparison of TDA Simplicial Complexes

Table 3. Topological Descriptors and Suitability for Transformer Training

Complex Type	Input Data	Key Construction Rule	Advantages	Disadvantages
Vietoris–Rips (RIPS)	Point cloud + distance matrix in any metric space.	Clique complex: A k -simplex for every subset of $k + 1$ points where all pairwise distances are $\leq r$.	Easy to define; widely applicable; fast implementations exist (e.g., Ripser).	Computationally expensive; number of simplices grows combinatorially/exponentially with points.
Alpha Complex	Point cloud in $\mathbb{R}^d$.	Subcomplex of the Delaunay triangulation restricted by a radius α .	Captures the true homotopy type of the union of balls; sparse; uses exact geometry.	Requires full Delaunay tessellation (even in 3D); restricted to Euclidean space.
Nerve Complex	Point cloud in $\mathbb{R}^d$.	A k -simplex if the intersection of $k + 1$ balls of radius r is non-empty.	Theoretically sound (Nerve Lemma); captures the true homotopy type of the union of balls.	Computationally expensive; difficult to materialize high-dimensional nerve structures.
Witness Complex	A large point cloud and a set of Landmarks L .	Approximates the Rips filtration by only considering proximity to landmarks.	Ultra-fast; scalable and memory efficient for billion-point clouds.	Coarse topological features; sensitive to the selection of landmarks.
Cubical Complex	Regular grid/voxel data (e.g., images).	Cells are hypercubes (pixels, voxels) based on scalar values.	Extremely efficient for structured data; ideal for medical imaging/CNN analysis on grids.	Only for structured grid data.

Recent Works in TDA for Neural Network Analysis The field of TDA for Neural Networks focuses on four broad domains: 1. Architecture, 2. Input/Output Spaces, 3. Internal Representations (Weights/Activations), and 4. Training Dynamics/Loss Functions. My project primarily relates to domains 3 and 4

The following table, similar to the survey provided, summarizes relevant recent contributions: Table 4: Summaries of Recent Works and Contributions

Title (Reference)	Summary / Contribution	Key Area / Cat.	Apps.
Differentiability and Optimization of Multiparameter Persistent Homology [69]	Developed a general theoretical framework proving that multiparameter persistence descriptors (like signed barcodes and persistence landscapes) are differentiable (semilinearly determined on grids), enabling their use in gradient descent pipelines.	TDA Theory / Optimization	-
Stable Vectorization of Multiparameter Persistent Homology using Signed Barcodes as Measures [66]	Introduced the Hilbert decomposition signed measure (μ_{Hil}) and rank decomposition signed measure as differentiable vectorizations of multiparameter persistence modules, offering stability guarantees.	Multiparameter TDA	-
Intrinsic dimension, persistent homology and generalization in neural networks [27]	Showed that the persistent homology dimension (a geometric fractal dimension derived from PDs) of the space of training weights correlates significantly with the generalization capacity of NNs (lower dimension = better generalization).	Training Dynamics (4)	-

Title (Reference)	Summary / Contribution	Key Area / Cat.	Apps.
A topological regularizer for regularizers via persistent homology (Chen et al., 2019) [6]	Proposed one of the first topological regularization terms for classifiers, using differentiable PH to remove weak connected components from the decision boundary, reducing overfitting. Regularization of neural networks by modifying their decision regions using differentiable persistent homology.	Loss Functions (4), (2)	(1)
Neural persistence: A complexity measure for deep neural networks using algebraic topology (Rieck et al., 2019) [10]	Defined Neural Persistence as a measure of topological complexity derived from the weights of FCFNNs, finding that higher persistence correlates with better generalization, usable as an early stopping criterion. Topological complexity measure for neural networks based on their weights.	Internal Rep. (3)	(1)
Representation topology divergence: A method for comparing neural network representations [40]	Proposed RTD, a measure to compare two Vietoris–Rips filtrations (representations) by counting how features merge, providing a dissimilarity measure between data representations.	Internal Rep. (3)	-
Topological dynamics of functional neural network graphs during reinforcement learning [67]	Studied the evolution of Betti numbers derived from complexes induced by functional neural network graphs during RL training/inference.	Internal Rep. (3)	-
On the complexity of neural network classifiers: A comparison between shallow and deep architectures (Bianchini and Scarselli, 2014) [80]	Bounds on the Betti numbers of the positive decision region generated by binary classification neural networks with Pfaffian activations.	(2)	-
Topological approaches to deep learning (Carlsson and Brumfiel Gabriellsson, 2020) [5]	Topological analysis of the weights of convolutional neural networks and generalization of convolutional neural networks.	(3)	-
Topological data analysis of decision boundaries with application to model selection (Ramamurthy et al., 2019) [97]	Study and approximation of the topology of network decision boundaries.	(2)	(5)
Geometry Score: A method for comparing generative adversarial networks (Khruikov and Oseledets, 2018) [9]	Measurement of GAN quality using persistent homology.	(2)	(7)
What does it mean to learn in deep networks? And, how does one detect adversarial attacks? (Corneanu et al., 2019) [15]	Study of generalization in terms of the persistent homology of neuron activations and their correlations.	(3)	(1, 3)
On connected sublevel sets in deep learning (Nguyen, 2019) [18]	Study of the connectivity, boundedness, and local minima of sublevel sets of convex losses for overparameterised neural networks with piecewise linear activation functions.	(4)	-
Characterizing the shape of activation space in deep neural networks (Gebhart et al., 2019) [17]	Topological characterization of the neuron activations given a sample.	(3)	(3)
Exposition and interpretation of the topology of neural networks (Brumfiel Gabriellsson and Carlsson, 2019) [16]	Topological analysis of the weights of convolutional neural networks and connection to generalization capacity of models.	(3)	-
Path homologies of deep feedforward networks (Chowdhury et al., 2019) [14]	Analysis of path and directed flag homology groups of MLPs' directed graph.	(1)	-
Topology of deep neural networks (Naitzat et al., 2020) [23]	Study of the topology of the data through layer transformations.	(3)	-
Finding the homology of decision boundaries with active learning (Li et al., 2020) [22]	Use of active learning to improve the methods of Topological data analysis of decision boundaries with application to model selection.	(2)	(5)
Computing the testing error without a testing set (Corneanu et al., 2020) [20]	Regression of test accuracy using topological vectorizations of persistence diagrams.	(3)	(6)
Topological detection of trojaned neural networks (Zheng et al., 2021) [38]	Study of trojaned networks in terms of the persistent homology of neuron activations and their correlations.	(3)	(4)
Experimental stability analysis of neural networks in classification problems with confidence sets for persistence diagrams (Akai et al., 2021) [26]	Study of persistence diagrams of last hidden layer activations and its connection with generalization.	(3)	-
PHom-GeM: Persistent homology for generative models (Charlier et al., 2019) [12]	Comparison between persistence diagrams of real and generated manifolds using generative models.	(2)	(7)
TopoAct: Visually exploring the shape of activations in deep learning (Rathore et al., 2021) [96]	Analysis of the Mapper graph of neuron activations for each layer.	(3)	-
Deep neural network pruning using persistent homology (Watanabe and Yamana, 2020) [25]	Pruning of neural networks using persistent homology.	(3)	(2)
Intrinsic dimension, persistent homology and generalization in neural networks (Birdal et al., 2021) [27]	Generalization error bounds using persistent homology dimension on the training weights.	(3, 4)	(1)
Activation landscapes as a topological summary of neural network performance (Wheeler et al., 2021) [35]	Study of the topology of the data through layer transformations and its connection with training accuracy.	(3)	-
Topology of learning in feedforward neural networks (Gabella, 2021) [28]	Analysis of the evolution of the weights of a neural network during training using Mapper.	(3)	-
Topological uncertainty: Monitoring trained neural networks through persistence of activation graphs (Lacombe et al., 2021) [29]	Uncertainty measurement of neural network predictions using the topology of neuron activations.	(3)	(3, 5)

Title (Reference)	Summary / Contribution	Key Area / Cat.	Apps.
Topological measurement of deep neural networks using persistent homology (Watanabe and Yamana, 2022b) [34]	Computation of persistence diagrams using neuron path relevance and connection with network expressivity and problem difficulty.	(3)	-
Evaluating the disentanglement of deep generative models with manifold topology (Zhou et al., 2021)[39]	Measurement of disentanglement of generative neural networks.	(2)	-
Quantitative performance assessment of CNN units via topological entropy calculation (Zhao and Zhang, 2022) [95]	Measurement of quality of convolutions in a neural network using persistent homology.	(3)	-
Overfitting measurement of deep neural networks using no data (Watanabe and Yamana, 2022a) [92]	Study of overfitting by analysing the points near the diagonal of a persistence diagram generated from the weights of a neural network.	(3)	-
An adversarial robustness perspective on the topology of neural networks (Goibert et al., 2022) [44]	Analysis of adversarial examples by means of the topology of the subgraph of under-optimized edges of neural networks.	(3)	(3)
Representation topology divergence: A method for comparing neural network representations (Baranikov et al., 2022) [41]	Definition of similarity between data representations using persistent homology.	(3)	(7)
On the topological expressive power of neural networks (Petri and Leita, 2020) [93]	Measurement of expressivity of network architectures.	(2)	-
Generalization bounds using data-dependent fractal dimensions (Dupuis et al., 2023)[53]	Data-dependent generalization error bounds using persistent homology dimension on the training weights.	(3, 4)	-
TopoBERT: Exploring the topology of fine-tuned word representations (Rathore et al., 2023) [58]	Mapper graph of transformer-based models with applications in finetuning of language models.	(3)	-
Experimental observations of the topology of convolutional neural network activations (Purvine et al., 2023) [49]	Analysis of the Mapper graph of neuron activations and definition of a similarity function between layers using sliced Wasserstein distances between persistence diagrams generated from them.	(3)	-
ReLU neural networks, polyhedral decompositions, and persistent homology (Liu et al., 2023b) [55]	Detection of homological features in manifolds embedded in the input space of a neural network using the polyhedra decomposition induced by a ReLU feedforward neural network.	(2)	-
Caveats of neural persistence in deep neural networks (Girrbach et al., 2023) [54]	Connection between neural persistence and variance measures of neural network weights.	(3)	-
On the use of persistent homology to control the generalization capacity of a neural network (Barbara et al., 2024) [91]	Regression of generalization gap using the average persistence of zero dimensional persistence diagrams.	(3)	(6)
Visualizing and analyzing the topology of neuron activations in deep adversarial training (Zhou et al., 2023) [99]	Analysis of the Mapper graph of neuron activations of normally and adversarially trained neural networks.	(3)	-
TopP&R: Robust Support Estimation Approach for Evaluating Fidelity and Diversity in Generative Models (Kim et al., 2023) [64]	Approximation of the precision and recall scores for generative models using a persistent homology-based approach.	(2)	(7)
Topological structure of complex predictions (Liu et al., 2023a) [46]	Analysis of the GTDA Reeb network of output spaces of neural networks.	(2)	-
Topological dynamics of functional neural network graphs during reinforcement learning (Muller et al., 2024)[67]	Study of reinforcement learning neural networks during training and inference using homology.	(3)	-
Topological dynamics of functional neural network graphs during reinforcement learning (Muller et al., 2024) [98]	Study of reinforcement learning neural networks during training and inference using homology.	(3)	-
Fast Witness Persistence for MRI Volumes via Hybrid Landmarking (Jorge Ruiz Williams 2025) [84]	The paper introduces a scalable GPU-accelerated method for computing persistent homology on full-brain MRI volumes using hybrid density-aware landmark selection and witness complexes, achieving runtime under 10 seconds while preserving topological features that traditional methods would take hours to compute.	-	-
A Framework for Fast and Stable Representations of Multiparameter Persistent Homology Decompositions (David Loiseaux et al 2023) [56]	-	-	-
Multi-parameter Module Approximation: an efficient and interpretable invariant for multi-parameter persistence modules with guarantees (David Loiseaux et al 2024) [47]	-	-	-
Stable Vectorization of Multiparameter Persistent Homology using Signed Barcodes as Measures (David Loiseaux et al 2023) [90]	-	-	-
Differentiability and Optimization of Multiparameter Persistent Homology (Luis Scoccola et al 2024) [70]	-	-	-

Table 3: Table containing the published articles reviewed in this survey ordered by year of publication. Each row corresponds to an article. The first column contains the title; the second column contains a one-line summary; the third column contains the categories introduced in Section 1 associated to each article; and the last column contains the applications introduced in Section 2.4. The four possible categories are 1. Structure of the neural network; 2. Input and output spaces; 3. Internal representations and activations; 4. Training dynamics and loss functions. The seven possible applications are 1. Regularization; 2. Pruning of neural networks; 3. Detection of adversarial, out-of-distribution and shifted examples; 4. Detection of trojaned networks; 5. Model selection; 6. Prediction of accuracy; 7. Quality assessment of generative models.

The Mapper algorithm

The breakthrough in utilizing two-dimensional (2D), or more generally multiparameter, persistent homology (PH) stems fundamentally from overcoming a major theoretical limitation of classical one-parameter PH, and subsequently developing differentiable descriptors that allow these rich invariants to be integrated into machine learning optimization tasks.

- **The Core Limitation of 1D Persistence** In classical persistent homology (often called one-parameter persistence), data is filtered along a single dimension, such as increasing radius (?) or density (?). This results in a sequence of nested spaces, known as a filtration. The key theoretical breakthrough for 1D persistence was the establishment of a complete discrete invariant, known as the persistence barcode (or persistence diagram). This barcode uniquely and completely encodes all the topological information within the filtration, up to isomorphism. However, many real-world structures are defined by multiple interacting parameters, leading to a multifiltration (e.g., parameterized along curvature ? and radius ?, or scale and density). If one attempts to use 1D PH on such a structure, one must fix the value of the other parameters, which risks losing valuable information or requiring arbitrary parameter choices.
- **The Negative Result (The Theoretical Wall)** The first step toward the 2D PH breakthrough was the critical realization by Gunnar Carlsson and Afra Zomorodian (2009) that the simplicity and completeness of the 1D barcode do not generalize to higher dimensions. **No Complete Discrete Invariant:** Due to fundamental algebraic reasons, no similar canonical complete discrete invariant (like the barcode) exists for n -valued filtering functions when $n > 1$ (e.g., for 2D persistence). The classification of multigraded modules (the algebraic structure corresponding to multifiltrations) involves both discrete and continuous portions, and the continuous portion has no precise, field-independent parameterization. **Field Dependence:** The classification in higher dimensions can be dependent on the choice of the coefficient field (k), meaning changing the field changes the classification and the target of the classification, preventing the existence of a complete discrete invariant. This negative result meant that new mathematical tools were needed to capture persistent topological features in n -parameter settings.
- **The Shift to Incomplete, but Useful, Invariants** Given the impossibility of a complete discrete invariant, the breakthrough shifted focus to developing invariants that are computable, compact, and effective for extracting persistent features—even if they are incomplete. The proposed alternatives measure the relationships (homomorphisms) between the filtered spaces, rather than just the birth/death times.

The Rank Invariant:

Carlsson and Zomorodian proposed the rank invariant (X, i) , a discrete invariant defined as the rank of the structure maps between homology groups. **Utility:** The rank invariant extends naturally to higher dimensions, remaining a homeomorphism invariant of the multifiltered space. In 1D, it is mathematically equivalent to the barcode, proving its completeness in that case. In 2D and higher, it is used to identify persistent features by looking for points far from the diagonal with a neighborhood of constant value.

Limitation: Unlike the barcode, the rank invariant is incomplete in higher dimensions. **B. Emergence of Differentiable Descriptors** A subsequent major practical breakthrough was developing methods to translate multiparameter PH structures into vectorizations that are differentiable for gradient-based optimization. This is crucial for integrating TDA topological insights into machine learning loss functions or layers.

1. **Signed Barcodes as Measures (Hilbert Decomposition Signed Measure):** These generalizations of the 1D barcode allow for representation in vector space. Works like Loiseaux et al. (2023b) developed techniques for stable vectorization of signed barcodes. 2. **Multiparameter Persistence Landscapes (MPPL):** This is a functional descriptor extending the 1D persistence landscape. 3. **Multiparameter Persistence Image (MPI):** This descriptor, which uses a convolution of a signed measure with a Gaussian kernel, is specifically designed for machine learning. 4. **Differentiability and Optimization: The Final Breakthrough** The definitive achievement that made 2D/multiparameter PH widely applicable in machine learning was providing a general, unified framework for differentiability and optimization.

Semilinear Determination on Grids: The paper "Differentiability and Optimization of Multiparameter Persistent Homology" (Scoccola et al., 2024) introduced a framework proving that a broad class of descriptors—those characterized as being semilinearly determined on grids—are semilinear maps with an explicit Clarke subdifferential.

This category encompasses powerful multiparameter invariants like signed barcodes as measures and the multiparameter persistence landscape.

Gradient Convergence Guarantees: This theoretical result generalizes the existing optimization framework for 1D barcodes and provides the sound mathematical foundation necessary for optimizing multi-parameter topological terms via gradient descent.

Improved Performance: Numerical experiments show that optimizing multiparameter homological descriptors tends to outperform their one-parameter counterparts in machine learning tasks. For instance, using a function-Rips bifiltration (metric + density) in optimization experiments prevents point clouds from diverging and preserves density-dependent structures, which single-parameter Rips optimization cannot achieve.

In summary, the breakthrough of using 2D persistent homology (multiparameter PH) was a two-step process: first, accepting the algebraic necessity of using incomplete invariants (like the rank invariant) instead of a complete barcode; and second, developing differentiable vectorizations and a comprehensive theoretical framework that allows these complex topological structures to be reliably and robustly optimized using standard gradient-based methods in deep learning.

5.5. Random Matrix Theory:

Random Matrix Theory (RMT) is a field of algebraic probability and mathematical physics that studies the properties of matrices whose entries are random variables, especially as the matrix dimensions become large. It is fundamentally a statistical approach: instead of analyzing the detailed laws of a complex system, RMT replaces deterministic matrices with random ones and calculates averages and other statistical properties of their eigenvalues and eigenvectors.

1. **Definition and Core Focus** A random matrix is simply a matrix whose elements are random variables.

RMT focuses on analyzing the statistical properties of these matrices, primarily their eigenvalues and eigenvectors. RMT is essential when dealing with systems too complicated for detailed study, such as the Hamiltonian of a heavy nucleus or complex financial data. Instead of attempting a deterministic prediction, RMT provides a statistical description. 2. **Historical Context and Applications** RMT gained attention primarily in nuclear physics. Wigner's Contribution: Eugene Wigner introduced random matrices to model the nuclei of heavy atoms. Wigner postulated that the spacing between energy levels (eigenvalues of the Hamiltonian) in complex systems should resemble the spacing between the eigenvalues of a random matrix, depending only on the symmetry class of the system. Wigner's Semicircle Law: Wigner showed that for large $N \rightarrow \infty$, the average spectral density $\rho(x)$ of many such matrices converges to the universal semicircle distribution. Wishart Matrices: John Wishart introduced random matrices to estimate covariance matrices in multivariate statistics, predating Wigner's physics applications. These matrices form the Wishart-Laguerre ensemble. RMT has wide applications, including: financial data analysis, quantum chaos (the BGS conjecture), computational neuroscience (modeling synaptic connections), and high-dimensional statistics.

3. **Key Concepts in RMT** A. **Ensembles and Symmetry Classes** Random matrices are grouped into ensembles based on the distribution of their entries and symmetry requirements. The most commonly studied are the Gaussian ensembles (GOE, GUE, GSE), categorized by the Dyson index (β): Gaussian

Orthogonal Ensemble (GOE): Real symmetric matrices ($\beta = 1$). Invariant under orthogonal conjugation. Models Hamiltonians with time-reversal symmetry. Gaussian Unitary Ensemble (GUE): Complex hermitian matrices ($\beta = 2$). Invariant under unitary conjugation. Models Hamiltonians lacking time-reversal symmetry. Gaussian Symplectic Ensemble (GSE): Hermitian quaternionic matrices ($\beta = 4$). Invariant under symplectic conjugation. The Dyson index β is equal to the number of real variables needed to specify one entry of the matrix.

B. The Joint Probability Density Function (JPDF) of Eigenvalues

For rotationally invariant ensembles, the JPDF of the eigenvalues, $\rho(x_1, \dots, x_N)$, contains two competing terms: **confinement** and **repulsion**.

$$\rho(x_1, \dots, x_N) = \frac{1}{Z} \left(\sum_{i=1}^N x_i^2 \right) \prod_{j < k} |x_j - x_k|^\beta \quad (\text{beta model})$$

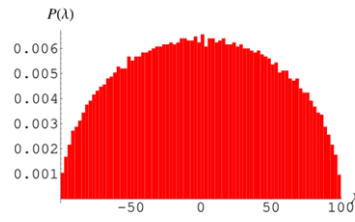
Confinement: The Gaussian exponential factor $\exp(-\frac{1}{2} \sum x_i^2)$ pushes eigenvalues toward the origin.

Repulsion: The Vandermonde determinant term $\prod_{j < k} |x_j - x_k|^\beta$ enforces level repulsion, meaning the probability of two eigenvalues being very close ($\delta \rightarrow 0$) is very small. This repulsion is raised to the power β , confirming that the strength of repulsion depends on the symmetry class.

C. Spectral Density and Limiting Laws

The spectral density $S\rho(x)$ (or average density of states) is the histogram profile of eigenvalues as $N \rightarrow \infty$. This is the marginal density of the JPDF $S\rho(x) = \int \rho(x, x_2, \dots, x_N) dx_2 \dots dx_N$.

- **Wigner's Semicircle Law:** For large Gaussian (Wigner) matrices, the rescaled spectral density converges to $S\rho(x) = \frac{4}{\pi} \sqrt{1 - x^2}$ over the interval $[-2, 2]$.



The distribution of eigenvalues of a symmetric random matrix with entries chosen from a standard normal distribution is illustrated above for a random 5000×5000 matrix.

Note that a large real symmetric matrix with random entries taken from a uniform distribution also obeys the semicircle law with the exception that it also possesses exactly one large eigenvalue.

Figure 29. Wigner's semi circle for Eigenvalues following gaussian noise

- **Marchenko–Pastur (MP) Law:** For Wishart-Laguerre matrices (rectangular), the limiting spectral density is the MP distribution $S\rho(y)$ with law characterized by the eigenvalue behavior of covariance matrices when N and M both tend to infinity while their aspect ratio $c = N/M \leq 1$ remains fixed.

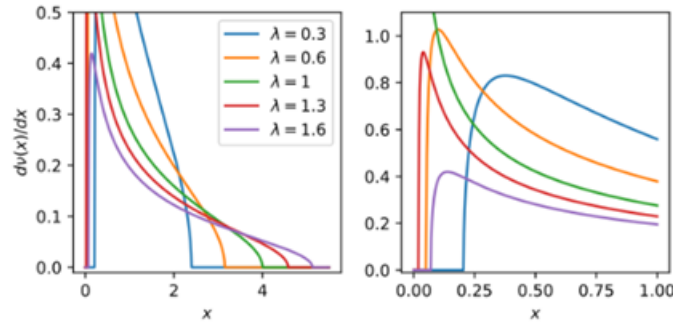


Figure 30. Marchenko–Pastur for non-symmetric matrix eigenvalues

The Spectrum Method in Neural Networks In the context of transformer networks, RMT is used through the spectrum method. This involves:

1. Treating layers (or groups of functionally similar layers, like query layers) as rectangular random matrices.
2. Computing the covariance matrix $\Sigma = W^T W / n$ of the layer weight matrix W and extracting its eigenvalues (or singular values).
3. Comparing the resulting empirical spectrum to the theoretical Marchenko-Pastur (MP) distribution
4. Interpretation: Eigenvalues lying within the MP bounds are interpreted as noise, suggesting their corresponding principal components carry little meaningful information, and thus the layer has lower importance

Part 2: Concepts to Retain

Given my thesis focus on weight regression and continual learning using Transformers, the most important RMT concepts to retain are those connecting matrix properties to information content and generalization:

- **Marchenko-Pastur Distribution as the Null Hypothesis** The Marchenko-Pastur (MP) distribution is the theoretical cornerstone for analyzing layer weights because they are typically represented as rectangular matrices or covariance matrices ($\Sigma = W^T W / n$).
Concept: The MP distribution defines the expected distribution of eigenvalues for a purely random covariance matrix.
Retention: Any eigenvalue of a weight matrix W that falls within the bounds predicted by the MP law ($\lambda \in [\lambda_-, \lambda_+]$) is considered consistent with noise. Conversely, eigenvalues outside these bounds represent signal or genuine structure, which is the information learned by the neural network. This provides a powerful metric for distinguishing useful model capacity from useless noise.

- Eigenvalue Repulsion vs. Feature Clustering RMT highlights that structured matrices exhibit level repulsion ($\prod |x_j - x_k|^\beta$), meaning eigenvalues are regularly spaced and avoid clustering .
Concept: In the context of neural network weights, features that are highly correlated (or functional units that are functionally redundant) would lead to clustered eigenvalues or zero eigenvalues in the covariance matrix. The strong repulsion observed in standard RMT ensembles suggests a fundamental structuring principle for matrices defined by independent entries (like GOE/GUE) .
Retention: Analyzing the eigenvalue spacing of my predicted weights could serve as a proxy for the structural robustness of the resulting model. Weights that are "too random" might exhibit the characteristic spacing of RMT models, whereas functional, learned weights (which often exhibit redundancy or compression) might deviate from these universality classes .
- The Resolvent Method and Spectral Density The resolvent or Stieltjes transform $G(z)$ is a key function used to analytically calculate the average spectral density $\rho(x)$ for large N , avoiding complex integral calculations .
Concept: The spectral density $\rho(x)$ can be recovered from the imaginary part of the resolvent $G(z)$ using the Sokhotski-Plemelj formula . For large N , the resolvent often satisfies a simple algebraic equation .
Retention: The analytic approach used in RMT (like solving for the resolvent via the Coulomb gas technique or matrix integration) demonstrates how global spectral properties (the shape of the $\rho(x)$) are derived from the local statistical properties of the matrix entries . This methodology could inspire techniques for analytically approximating the spectral distribution of the Transformer-generated weight matrices, thus giving insight into the capacity and scale of the generated hyper-representations .
- Eigenvector Localization (IPR) and Mobility Edges The concept of eigenvector localization is measured by the Inverse Participation Ratio (IPR), $I_{N,j}$. Concept: IPR quantifies how "spread out" an eigenvector is: $I_{N,j} \approx 1/N$ means the state is extended (all components are relevant), while $I_{N,j} \approx 1/s$ (for small s) means the state is localized (only a few components are active)
Retention: The localization properties of the eigenvectors associated with the weights could correlate strongly with the redundancy, efficiency, or modularity of the generated CNNs. A low IPR for the principal components (extended states) might indicate efficient use of capacity, whereas highly localized states might suggest specific, compartmentalized knowledge within the network, potentially useful for understanding which weights are finetunable . The concept of a mobility edge (the critical eigenvalue separating extended/localized states) is particularly valuable for this analysis .

5.6. (

kolmogorov arnold networks KAN)

[79] Since the diffusion paper shows that finetuning is needed for CNNs to have good performance . we decide to keep weights before and after finetuning and since it's a process that happens extremely fast we assume the underlying function is simple . simple enough to be approximated by Kolmogorov arnold networks . The theory supporting my approach—approximating the fast, functionally simple weight transformation during fine-tuning using Kolmogorov-Arnold Networks (KANs)—rests on the Kolmogorov-Arnold Representation Theorem (KAT), the demonstrated efficiency of hyper-representation models like D2NWG and Hyper-Representations, and insights into the simplicity and manifold structure of neural network solutions. Here is a detailed explanation of the theoretical basis that could make this work, drawing specifically on the sources provided.

The Theoretical Foundation: KANs for Weight Transformation My core assumption is:

1. The performance gap between predicted weights ($W_{\text{predicted}}$) and high-performing fine-tuned weights ($W_{\text{finetuned}}$) is bridged quickly during fine-tuning (e.g., in one epoch).
2. This fast transformation implies the underlying function mapping $W_{\text{predicted}}$ to $W_{\text{finetuned}}$ is relatively simple (or low-dimensional).
3. This simple transformation function can be effectively approximated by a Kolmogorov-Arnold Network (KAN).
4. Kolmogorov-Arnold Representation Theorem (KAT) The primary mathematical theory underpinning the KAN architecture is the Kolmogorov-Arnold Representation Theorem (KAT). The Theorem: KAT states that any continuous multivariate function $f(x_1, \dots, x_n)$ on a bounded domain can be represented as a finite superposition of continuous functions of a single variable and the binary operation of addition.

$$f(x) = \sum_{q=1}^{2n+1} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right)$$

The KAN Architecture: The KAN architecture directly embodies this theorem, using learnable activation functions (often parametrized by B-splines) on the edges, rather than fixed activation functions (like ReLU) on the nodes as in a traditional Multi-Layer Perceptron (MLP). The functions $\phi_{q,p}(x_p)$ and Φ_q are the 1D activation functions learned on edges. Simplicity and Interpretability: KANs are proposed as alternatives to MLPs, empirically showing they can achieve better accuracy, faster numerical scaling laws, and greater interpretability in terms of accuracy and interpretability trade-offs on various science tasks. For problems where the underlying function is structurally simple (e.g., following a known symbolic formula), KANs can discover this exact structure automatically.

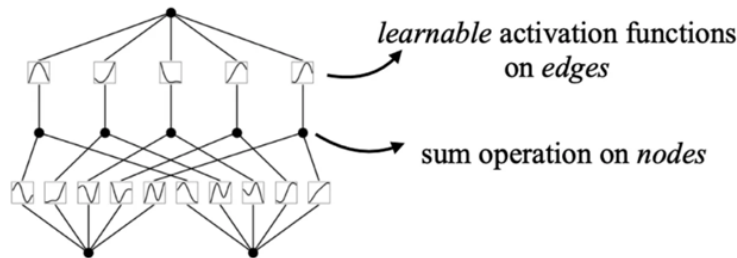


Figure 31. KAN models learn activation functions that minimize total loss

- Application to Weight Transformation: You hypothesize that the function mapping the generated weights (W) to the high-performing, fine-tuned weights is simple. This simplicity is evidenced by the fast convergence observed when fine-tuning generated hyper-representations: Fast Fine-Tuning: Weights generated by models like D2NWG or SKDE often accelerate convergence, achieving superior performance in a few epochs (e.g., 25 epochs) compared to training from random initialization (e.g., 50 epochs). The need for only a couple hundred steps (essentially a single epoch) to reach desirable performance confirms the speed of this transformation. Simplicity Implication: This suggests that the generated weights $W_{\text{predicted}}$ are already located on or very close to the low-dimensional manifold of high-performing weights. The quick fine-tuning step is merely a short, simple trajectory that moves the weights from the predicted point on the manifold to a nearby optimal local minima W^* .
KAN as the Simple Map: Since the required correction is rapid and localized, the functional difference $F(W_{\text{predicted}}) = W_{\text{finetuned}}$ might be simple enough to be effectively captured by a KAN. KANs excel at fitting functions where the intrinsic dimensionality is low or the function has a compact

- Connection to Loss Landscape and Manifold Structure The success of this approximation relies on the idea that the "simple function" F exists because the weights lie on a structured manifold. Weight Manifold: Neural network models populate a low dimensional manifold. Weights generated by hyper-representation models like D2NWG are hypothesized to lie on the same manifold as the original and fine-tuned weights, enabling the method to produce optimized weights without needing extensive fine-tuning. Loss Flatness and Simple Curves: The underlying weight transformation during fine-tuning likely follows simple curves connecting different optima. The final predicted weights $W_{\text{predicted}}$ are already placed in a region of the loss surface that leads to faster learning and better solutions. Your previous observation confirmed that MSE increases post-fine-tuning, but the Wasserstein Distance (WD) decreases meaningfully, suggesting the fine-tuning jump is geometrically effective, moving from a sub-optimal point $W_{\text{predicted}}$ to a functionally equivalent point $W_{\text{finetuned}}$ that resides on a flat loss region. KAN for Gradient-Free/Interpretable Layers: If the KAN is used to model the dense layer merging the embeddings or as a post-processor to refine the predicted weights, it introduces interpretable, non-gradient-based layers with fewer parameters. This aligns with your overall goal of creating a model that is more explainable and efficient.
- Technical Feasibility While KANs were initially presented as being interpretable and efficient for function fitting, their integration into a deep learning pipeline requires differentiability, which is supported by their implementation: Differentiable Operations: The KAN architecture, particularly the standard composition of L layers $\text{KAN}(x) = \sum_i \text{splines}_i(x) \cdot \text{weights}_i$, is built on basic operations (summation, multiplication) and learnable activation functions parametrized by B-splines. Since all the operations are differentiable, KANs can be trained via backpropagation. This makes them theoretically suitable for integration into your Transformer's weight regression loss calculation, potentially as a differentiable refinement stage.

References

- [1] Y. Brenier, “Polar factorization and monotone rearrangement of vector-valued functions”, *Communications on Pure and Applied Mathematics*, vol. 44, no. 4, pp. 375–417, Jun. 1991, ISSN: 0010-3640, 1097-0312. DOI: [10.1002/cpa.3160440402](#) Accessed: Nov. 12, 2025.
- [2] M.-T. Luong, H. Pham, and C. D. Manning, *Effective Approaches to Attention-based Neural Machine Translation*, Sep. 2015. DOI: [10.48550/arXiv.1508.04025](#) arXiv: [1508.04025 \[cs\]](#). Accessed: Nov. 24, 2025.
- [3] D. Bahdanau, K. Cho, and Y. Bengio, *Neural Machine Translation by Jointly Learning to Align and Translate*, May 2016. DOI: [10.48550/arXiv.1409.0473](#) arXiv: [1409.0473 \[cs\]](#). Accessed: Nov. 24, 2025.
- [4] H. Shin, J. K. Lee, J. Kim, and J. Kim, *Continual Learning with Deep Generative Replay*, Dec. 2017. DOI: [10.48550/arXiv.1705.08690](#) arXiv: [1705.08690 \[cs\]](#). Accessed: Sep. 20, 2024.
- [5] G. Carlsson and R. B. Gabrielsson, *Topological Approaches to Deep Learning*, Nov. 2018. DOI: [10.48550/arXiv.1811.01122](#) arXiv: [1811.01122 \[cs\]](#). Accessed: Nov. 24, 2025.
- [6] C. Chen, X. Ni, Q. Bai, and Y. Wang, *A Topological Regularizer for Classifiers via Persistent Homology*, Oct. 2018. DOI: [10.48550/arXiv.1806.10714](#) arXiv: [1806.10714 \[cs\]](#). Accessed: Nov. 16, 2025.
- [7] N. Díaz-Rodríguez, V. Lomonaco, D. Filliat, and D. Maltoni, *Don't forget, there is more than forgetting: New metrics for Continual Learning*, Oct. 2018. DOI: [10.48550/arXiv.1810.13166](#) arXiv: [1810.13166 \[cs\]](#). Accessed: Nov. 16, 2025.
- [8] D. Hallac, S. Vare, S. Boyd, and J. Leskovec, *Toeplitz Inverse Covariance-Based Clustering of Multivariate Time Series Data*, May 2018. DOI: [10.48550/arXiv.1706.03161](#) arXiv: [1706.03161 \[cs\]](#). Accessed: Nov. 16, 2025.
- [9] V. Khrulkov and I. Oseledets, *Geometry Score: A Method For Comparing Generative Adversarial Networks*, Jun. 2018. DOI: [10.48550/arXiv.1802.02664](#) arXiv: [1802.02664 \[cs\]](#). Accessed: Nov. 16, 2025.
- [10] B. Rieck et al., “Neural Persistence: A Complexity Measure for Deep Neural Networks Using Algebraic Topology”, in *International Conference on Learning Representations*, Sep. 2018. Accessed: Nov. 16, 2025.
- [11] P. Shaw, J. Uszkoreit, and A. Vaswani, *Self-Attention with Relative Position Representations*, Apr. 2018. DOI: [10.48550/arXiv.1803.02155](#) arXiv: [1803.02155 \[cs\]](#). Accessed: Nov. 24, 2025.
- [12] J. Charlier, R. State, and J. Hilger, *PHom-GeM: Persistent Homology for Generative Models*, May 2019. DOI: [10.48550/arXiv.1905.09894](#) arXiv: [1905.09894 \[cs\]](#). Accessed: Nov. 16, 2025.
- [13] R. Child, S. Gray, A. Radford, and I. Sutskever, *Generating Long Sequences with Sparse Transformers*, Apr. 2019. DOI: [10.48550/arXiv.1904.10509](#) arXiv: [1904.10509 \[cs\]](#). Accessed: Nov. 24, 2025.
- [14] S. Chowdhury, T. Gebhart, S. Huntsman, and M. Yutin, “Path homologies of deep feedforward networks”, in *2019 18th IEEE International Conference On Machine Learning And Applications (ICMLA)*, Dec. 2019, pp. 1077–1082. DOI: [10.1109/ICMLA.2019.00181](#) arXiv: [1910.07617 \[math\]](#). Accessed: Nov. 16, 2025.
- [15] C. A. Corneanu, M. Madadi, S. Escalera, and A. M. Martinez, “What Does It Mean to Learn in Deep Networks? And, How Does One Detect Adversarial Attacks?”, in *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Long Beach, CA, USA: IEEE, Jun. 2019, pp. 4752–4761, ISBN: 978-1-7281-3293-8. DOI: [10.1109/CVPR.2019.00489](#) Accessed: Nov. 16, 2025.
- [16] R. B. Gabrielsson and G. Carlsson, *Exposition and Interpretation of the Topology of Neural Networks*, Oct. 2019. DOI: [10.48550/arXiv.1810.03234](#) arXiv: [1810.03234 \[cs\]](#). Accessed: Sep. 20, 2024.
- [17] T. Gebhart, P. Schrater, and A. Hylton, *Characterizing the Shape of Activation Space in Deep Neural Networks*, May 2019. DOI: [10.48550/arXiv.1901.09496](#) arXiv: [1901.09496 \[cs\]](#). Accessed: Nov. 16, 2025.
- [18] Q. Nguyen, *On Connected Sublevel Sets in Deep Learning*, May 2019. DOI: [10.48550/arXiv.1901.07417](#) arXiv: [1901.07417 \[cs\]](#). Accessed: Nov. 16, 2025.
- [19] I. Beltagy, M. E. Peters, and A. Cohan, *Longformer: The Long-Document Transformer*, Dec. 2020. DOI: [10.48550/arXiv.2004.05150](#) arXiv: [2004.05150 \[cs\]](#). Accessed: Nov. 16, 2025.
- [20] C. Corneanu, M. Madadi, S. Escalera, and A. Martinez, *Computing the Testing Error without a Testing Set*, May 2020. DOI: [10.48550/arXiv.2005.00450](#) arXiv: [2005.00450 \[cs\]](#). Accessed: Nov. 16, 2025.
- [21] N. Kitaev, . Kaiser, and A. Levskaya, *Reformer: The Efficient Transformer*, Feb. 2020. DOI: [10.48550/arXiv.2001.04451](#) arXiv: [2001.04451 \[cs\]](#). Accessed: Nov. 16, 2025.
- [22] W. Li, G. Dasarthy, K. N. Ramamurthy, and V. Berisha, *Finding the Homology of Decision Boundaries with Active Learning*, Nov. 2020. DOI: [10.48550/arXiv.2011.09645](#) arXiv: [2011.09645 \[cs\]](#). Accessed: Nov. 16, 2025.
- [23] G. Naitzat, A. Zhitnikov, and L.-H. Lim, *Topology of deep neural networks*, Apr. 2020. DOI: [10.48550/arXiv.2004.06093](#) arXiv: [2004.06093 \[cs\]](#). Accessed: Nov. 16, 2025.
- [24] S. Wang, B. Z. Li, M. Khabsa, H. Fang, and H. Ma, *Linformer: Self-Attention with Linear Complexity*, Jun. 2020. DOI: [10.48550/arXiv.2006.04768](#) arXiv: [2006.04768 \[cs\]](#). Accessed: Nov. 16, 2025.
- [25] S. Watanabe and H. Yamana, “Deep Neural Network Pruning Using Persistent Homology”, in *2020 IEEE Third International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, Dec. 2020, pp. 153–156. DOI: [10.1109/AIKE48582.2020.00030](#) Accessed: Nov. 16, 2025.
- [26] N. Akai, T. Hirayama, and H. Murase, “Experimental stability analysis of neural networks in classification problems with confidence sets for persistence diagrams”, *Neural Networks*, vol. 143, pp. 42–51, Nov. 2021, ISSN: 0893-6080. DOI: [10.1016/j.neunet.2021.05.007](#) Accessed: Nov. 16, 2025.
- [27] T. Birdal, A. Lou, L. Guibas, and U. Imekli, *Intrinsic Dimension, Persistent Homology and Generalization in Neural Networks*, Nov. 2021. DOI: [10.48550/arXiv.2111.13171](#) arXiv: [2111.13171 \[cs\]](#). Accessed: Nov. 16, 2025.
- [28] M. Gabella, “Topology of Learning in Feedforward Neural Networks”, *IEEE Transactions on Neural Networks and Learning Systems*, vol. 32, no. 8, pp. 3588–3592, Aug. 2021, ISSN: 2162-2388. DOI: [10.1109/TNNLS.2020.3015790](#) Accessed: Nov. 16, 2025.
- [29] T. Lacombe, Y. Ike, M. Carriere, F. Chazal, M. Glisse, and Y. Umeda, *Topological Uncertainty: Monitoring trained neural networks through persistence of activation graphs*, May 2021. DOI: [10.48550/arXiv.2105.04404](#) arXiv: [2105.04404 \[stat\]](#). Accessed: Nov. 16, 2025.
- [30] L. Liu, J. Liu, and J. Han, *Multi-head or Single-head? An Empirical Comparison for Transformer Training*, Jun. 2021. DOI: [10.48550/arXiv.2106.09650](#) arXiv: [2106.09650 \[cs\]](#). Accessed: Nov. 24, 2025.
- [31] A. Rosasco, A. Carta, A. Cossu, V. Lomonaco, and D. Bacciu, *Distilled Replay: Overcoming Forgetting through Synthetic Samples*, Jun. 2021. arXiv: [2103.15851 \[cs\]](#). Accessed: Sep. 16, 2024.

- [32] K. Schürholt, D. Kostadinov, and D. Borth, “Self-Supervised Representation Learning on Neural Network Weights for Model Characteristic Prediction”, in *Advances in Neural Information Processing Systems*, vol. 34, Curran Associates, Inc., 2021, pp. 16 481–16 493. Accessed: Sep. 20, 2024.
- [33] Stanford Online, *Stanford CS224N NLP with Deep Learning | Winter 2021 | Lecture 9 - Self-Attention and Transformers*, Oct. 2021. Accessed: Nov. 24, 2025.
- [34] S. Watanabe and H. Yamana, *Topological Measurement of Deep Neural Networks Using Persistent Homology*, Jun. 2021. DOI: [10.48550/arXiv.2106.03016](#) arXiv: [2106.03016 \[cs\]](#). Accessed: Nov. 16, 2025.
- [35] M. Wheeler, J. Bouza, and P. Bubenik, “Activation Landscapes as a Topological Summary of Neural Network Performance”, in *2021 IEEE International Conference on Big Data (Big Data)*, Dec. 2021, pp. 3865–3870. DOI: [10.1109/BigData52589.2021.9671368](#) arXiv: [2110.10136 \[cs\]](#). Accessed: Nov. 16, 2025.
- [36] J. Yoon, W. Jeong, G. Lee, E. Yang, and S. J. Hwang, *Federated Continual Learning with Weighted Inter-client Transfer*, Jun. 2021. DOI: [10.48550/arXiv.2003.03196](#) arXiv: [2003.03196 \[cs\]](#). Accessed: Nov. 16, 2025.
- [37] M. Zaheer et al., *Big Bird: Transformers for Longer Sequences*, Jan. 2021. DOI: [10.48550/arXiv.2007.14062](#) arXiv: [2007.14062 \[cs\]](#). Accessed: Nov. 16, 2025.
- [38] S. Zheng, Y. Zhang, H. Wagner, M. Goswami, and C. Chen, *Topological Detection of Trojaned Neural Networks*, Jun. 2021. DOI: [10.48550/arXiv.2106.06469](#) arXiv: [2106.06469 \[cs\]](#). Accessed: Nov. 16, 2025.
- [39] S. Zhou, E. Zelikman, F. Lu, A. Y. Ng, G. Carlsson, and S. Ermon, *Evaluating the Disentanglement of Deep Generative Models through Manifold Topology*, Mar. 2021. DOI: [10.48550/arXiv.2006.03680](#) arXiv: [2006.03680 \[stat\]](#). Accessed: Nov. 24, 2025.
- [40] S. Barannikov, I. Trofimov, N. Balabin, and E. Burnaev, *Representation Topology Divergence: A Method for Comparing Neural Network Representations*, Jun. 2022. DOI: [10.48550/arXiv.2201.00058](#) arXiv: [2201.00058 \[cs\]](#). Accessed: Nov. 16, 2025.
- [41] S. Barannikov, I. Trofimov, N. Balabin, and E. Burnaev, *Representation Topology Divergence: A Method for Comparing Neural Network Representations*, Jun. 2022. DOI: [10.48550/arXiv.2201.00058](#) arXiv: [2201.00058 \[cs\]](#). Accessed: Nov. 16, 2025.
- [42] K. Choromanski et al., *Rethinking Attention with Performers*, Nov. 2022. DOI: [10.48550/arXiv.2009.14794](#) arXiv: [2009.14794 \[cs\]](#). Accessed: Nov. 16, 2025.
- [43] W. Fedus, B. Zoph, and N. Shazeer, *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*, Jun. 2022. DOI: [10.48550/arXiv.2101.03961](#) arXiv: [2101.03961 \[cs\]](#). Accessed: Nov. 16, 2025.
- [44] M. Goibert, T. Ricatte, and E. Dohmatob, *An Adversarial Robustness Perspective on the Topology of Neural Networks*, Nov. 2022. DOI: [10.48550/arXiv.2211.02675](#) arXiv: [2211.02675 \[cs\]](#). Accessed: Nov. 16, 2025.
- [45] J. Lee-Thorp, J. Ainslie, I. Eckstein, and S. Ontanon, *FNet: Mixing Tokens with Fourier Transforms*, May 2022. DOI: [10.48550/arXiv.2105.03824](#) arXiv: [2105.03824 \[cs\]](#). Accessed: Nov. 16, 2025.
- [46] M. Liu, T. K. Dey, and D. F. Gleich, *Topological structure of complex predictions*, Oct. 2022. DOI: [10.48550/arXiv.2207.14358](#) arXiv: [2207.14358 \[cs\]](#). Accessed: Nov. 16, 2025.
- [47] D. Loiseaux, M. Carrière, and A. J. Blumberg, *Multi-parameter Module Approximation: An efficient and interpretable invariant for multi-parameter persistence modules with guarantees*, 2022. DOI: [10.48550/ARXIV.2206.02026](#) Accessed: Nov. 12, 2025.
- [48] M. Mundt, S. Lang, Q. Delfosse, and K. Kersting, *CLEVA-Compass: A Continual Learning Evaluation Assessment Compass to Promote Research Transparency and Comparability*, Feb. 2022. DOI: [10.48550/arXiv.2110.03331](#) arXiv: [2110.03331 \[cs\]](#). Accessed: Nov. 16, 2025.
- [49] E. Purvine et al., *Experimental Observations of the Topology of Convolutional Neural Network Activations*, Dec. 2022. DOI: [10.48550/arXiv.2212.00222](#) arXiv: [2212.00222 \[cs\]](#). Accessed: Nov. 16, 2025.
- [50] Z. Qin et al., *cosFormer: Rethinking Softmax in Attention*, Feb. 2022. DOI: [10.48550/arXiv.2202.08791](#) arXiv: [2202.08791 \[cs\]](#). Accessed: Nov. 16, 2025.
- [51] K. Schürholt, B. Knyazev, X. Giró-i-Nieto, and D. Borth, *Hyper-Representations for Pre-Training and Transfer Learning*, Jul. 2022. DOI: [10.48550/arXiv.2207.10951](#) arXiv: [2207.10951 \[cs\]](#). Accessed: Sep. 20, 2024.
- [52] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints*, Dec. 2023. DOI: [10.48550/arXiv.2305.13245](#) arXiv: [2305.13245 \[cs\]](#). Accessed: Nov. 24, 2025.
- [53] B. Dupuis, G. Deligiannidis, and U. Imekli, *Generalization Bounds with Data-dependent Fractal Dimensions*, Jul. 2023. DOI: [10.48550/arXiv.2302.02766](#) arXiv: [2302.02766 \[stat\]](#). Accessed: Nov. 24, 2025.
- [54] L. Girrbaach, A. Christensen, O. Winther, Z. Akata, and A. S. Koepke, *Addressing caveats of neural persistence with deep graph persistence*, Nov. 2023. DOI: [10.48550/arXiv.2307.10865](#) arXiv: [2307.10865 \[cs\]](#). Accessed: Nov. 16, 2025.
- [55] Y. Liu, C. M. Cole, C. Peterson, and M. Kirby, *ReLU Neural Networks, Polyhedral Decompositions, and Persistent Homology*, Jun. 2023. DOI: [10.48550/arXiv.2306.17418](#) arXiv: [2306.17418 \[math\]](#). Accessed: Nov. 16, 2025.
- [56] D. Loiseaux, M. Carrière, and A. J. Blumberg, *A Framework for Fast and Stable Representations of Multiparameter Persistent Homology Decompositions*, Jun. 2023. DOI: [10.48550/arXiv.2306.11170](#) arXiv: [2306.11170 \[cs\]](#). Accessed: Nov. 16, 2025.
- [57] D. Qi, H. Zhao, and S. Li, *Better Generative Replay for Continual Federated Learning*, Feb. 2023. arXiv: [2302.13001 \[cs\]](#). Accessed: Sep. 20, 2024.
- [58] A. Rathore, Y. Zhou, V. Srikumar, and B. Wang, “TopoBERT: Exploring the topology of fine-tuned word representations”, *Information Visualization*, vol. 22, no. 3, pp. 186–208, Jul. 2023, ISSN: 1473-8716, 1473-8724. DOI: [10.1177/14738716231168671](#) Accessed: Nov. 16, 2025.
- [59] A. Vaswani et al., *Attention Is All You Need*, Aug. 2023. DOI: [10.48550/arXiv.1706.03762](#) arXiv: [1706.03762 \[cs\]](#). Accessed: Nov. 24, 2025.
- [60] R. Ballester, C. Casacuberta, and S. Escalera, *Topological Data Analysis for Neural Network Analysis: A Comprehensive Survey*, Jan. 2024. DOI: [10.48550/arXiv.2312.05840](#) arXiv: [2312.05840 \[cs\]](#). Accessed: Nov. 12, 2025.
- [61] T. Dao and A. Gu, *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality*, May 2024. DOI: [10.48550/arXiv.2405.21060](#) arXiv: [2405.21060 \[cs\]](#). Accessed: Nov. 16, 2025.
- [62] S. De et al., *Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models*, Feb. 2024. DOI: [10.48550/arXiv.2402.19427](#) arXiv: [2402.19427 \[cs\]](#). Accessed: Nov. 16, 2025.
- [63] A. Gu and T. Dao, *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*, May 2024. DOI: [10.48550/arXiv.2312.00752](#) arXiv: [2312.00752 \[cs\]](#). Accessed: Nov. 16, 2025.
- [64] P. J. Kim, Y. Jang, J. Kim, and J. Yoo, *TopP&R: Robust Support Estimation Approach for Evaluating Fidelity and Diversity in Generative Models*, Jan. 2024. DOI: [10.48550/arXiv.2306.08013](#) arXiv: [2306.08013 \[cs\]](#). Accessed: Nov. 16, 2025.
- [65] O. Lieber et al., *Jamba: A Hybrid Transformer-Mamba Language Model*, Jul. 2024. DOI: [10.48550/arXiv.2403.19887](#) arXiv: [2403.19887 \[cs\]](#). Accessed: Nov. 16, 2025.

- [66] D. Loiseaux, L. Scoccola, M. Carrière, M. B. Botnan, and S. Oudot, *Stable Vectorization of Multiparameter Persistent Homology using Signed Barcodes as Measures*, Feb. 2024. DOI: [10.48550/arXiv.2306.03801](#) arXiv: [2306.03801](#) [cs]. Accessed: Nov. 16, 2025.
- [67] M. Muller, S. Kroon, and S. Chalup, “Topological Dynamics of Functional Neural Network Graphs During Reinforcement Learning”, in *Neural Information Processing*, B. Luo, L. Cheng, Z.-G. Wu, H. Li, and C. Li, Eds., Singapore: Springer Nature, 2024, pp. 190–204, ISBN: 978-981-99-8138-0. DOI: [10.1007/978-981-99-8138-0_16](#)
- [68] K. Schürholt, *Hyper-Representations: Learning from Populations of Neural Networks*, Oct. 2024. DOI: [10.48550/arXiv.2410.05107](#) arXiv: [2410.05107](#) [cs]. Accessed: Nov. 16, 2025.
- [69] L. Scoccola, S. Setlur, D. Loiseaux, M. Carrière, and S. Oudot, *Differentiability and Optimization of Multiparameter Persistent Homology*, Aug. 2024. DOI: [10.48550/arXiv.2406.07224](#) arXiv: [2406.07224](#) [cs]. Accessed: Nov. 16, 2025.
- [70] L. Scoccola, S. Setlur, D. Loiseaux, M. Carrière, and S. Oudot, *Differentiability and Optimization of Multiparameter Persistent Homology*, Aug. 2024. DOI: [10.48550/arXiv.2406.07224](#) arXiv: [2406.07224](#) [cs]. Accessed: Nov. 16, 2025.
- [71] J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani, and T. Dao, *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision*, Jul. 2024. DOI: [10.48550/arXiv.2407.08608](#) arXiv: [2407.08608](#) [cs]. Accessed: Nov. 24, 2025.
- [72] B. Soro et al., *Diffusion-Based Neural Network Weights Generation*, Oct. 2024. DOI: [10.48550/arXiv.2402.18153](#) arXiv: [2402.18153](#) [cs]. Accessed: Nov. 12, 2025.
- [73] M. Vladymyrov, A. Zhmoginov, and M. Sandler, *Continual HyperTransformer: A Meta-Learner for Continual Few-Shot Learning*, Aug. 2024. DOI: [10.48550/arXiv.2301.04584](#) arXiv: [2301.04584](#) [cs]. Accessed: Sep. 20, 2024.
- [74] L. Wang, X. Zhang, H. Su, and J. Zhu, *A Comprehensive Survey of Continual Learning: Theory, Method and Application*, Feb. 2024. DOI: [10.48550/arXiv.2302.00487](#) arXiv: [2302.00487](#) [cs]. Accessed: Nov. 16, 2025.
- [75] Z. Fu et al., *Sliding Window Attention Training for Efficient Large Language Models*, Feb. 2025. DOI: [10.48550/arXiv.2502.18845](#) arXiv: [2502.18845](#) [cs]. Accessed: Nov. 24, 2025.
- [76] *GUDHI/TDA-tutorial*, Geometry Understanding in Higher Dimensions, Nov. 2025. Accessed: Nov. 12, 2025.
- [77] *Hyper-Representations as Generative Models: Sampling Unseen Neural Network Weights P63172 | GTC 2024 | NVIDIA On-Demand*, <https://www.nvidia.com/en-us/on-demand/session/gtc24-p63172/>, Nov. 2025. Accessed: Nov. 13, 2025.
- [78] B. Knyazev, A. Moudgil, G. Lajoie, E. Belilovsky, and S. Lacoste-Julien, *Accelerating Training with Neuron Interaction and Nowcasting Networks*, Feb. 2025. DOI: [10.48550/arXiv.2409.04434](#) arXiv: [2409.04434](#) [cs]. Accessed: Nov. 24, 2025.
- [79] Z. Liu et al., *KAN: Kolmogorov-Arnold Networks*, Feb. 2025. DOI: [10.48550/arXiv.2404.19756](#) arXiv: [2404.19756](#) [cs]. Accessed: Nov. 16, 2025.
- [80] “On the Complexity of Neural Network Classifiers: A Comparison Between Shallow and Deep Architectures | Request PDF”, *ResearchGate*, Aug. 2025. DOI: [10.1109/TNNLS.2013.2293637](#) Accessed: Nov. 16, 2025.
- [81] *Optimal Transport - Weak Solutions of Monge-Ampere - YouTube*, <https://www.youtube.com/watch?v=mmpyxStCoqU&list=PLJ6garKOLK2qKVhRm6UwvcQ46wK-ciHbl&index=13>, Nov. 2025. Accessed: Nov. 12, 2025.
- [82] *README.md · ReactiveAI/MQA-Ref-Micro at main*, <https://huggingface.co/ReactiveAI/MQA-Ref-Micro/blob/main/README.md>, Oct. 2025. Accessed: Nov. 24, 2025.
- [83] L. Wang, *Lywang3081/Awesome-Continual-Learning*, Oct. 2025. Accessed: Nov. 16, 2025.
- [84] J. L. R. Williams, *Fast Witness Persistence for MRI Volumes via Hybrid Landmarking*, Oct. 2025. DOI: [10.48550/arXiv.2510.04553](#) arXiv: [2510.04553](#) [cs]. Accessed: Nov. 12, 2025.
- [85] C. Wu, M. Che, R. Xu, Z. Ran, and H. Yan, “ELFATT: Efficient Linear Fast Attention for Vision Transformers”, in *Proceedings of the 33rd ACM International Conference on Multimedia*, Dublin Ireland: ACM, Oct. 2025, pp. 9140–9149, ISBN: 979-8-4007-2035-2. DOI: [10.1145/3746027.3754825](#) Accessed: Nov. 24, 2025.
- [86] *(PDF) Hidden Markov Model Regression*, https://www.researchgate.net/publication/2791408_Hidden_Markov_Model_Regression. Accessed: Nov. 16, 2025.
- [87] *Avalanche: An End-to-End Library for Continual Learning | Avalanche*, <https://avalanche.continualai.org>. Accessed: Nov. 16, 2025.
- [88] 2. v. D. m. l. 1. déc. 2019, *Optimal Transport*, <http://www.youtube.com/playlist?list=PLJ6garKOLK2qKVhRm6UwvcQ46wK-ciHbl>. Accessed: Nov. 24, 2025.
- [89] 1. v. D. m. l. 6. janv. 2022, *Continual Learning Course*, <http://www.youtube.com/playlist?list=PLm6QXeaB-XkBfM5RgQP6wCR7Jegdg51Px>. Accessed: Nov. 24, 2025.
- [90] D. Loiseaux, L. Scoccola, M. Carrière, M. B. Botnan, and S. Oudot, “Stable Vectorization of Multiparameter Persistent Homology using Signed Barcodes as Measures”,
- [91] *On the Use of Persistent Homology to Control the Generalization Capacity of a Neural Network*, <http://ouci.dntb.gov.ua/en/works/4zn1VeE4/>. Accessed: Nov. 16, 2025.
- [92] “Overfitting Measurement of Deep Neural Networks Using No Data | Request PDF”, in *ResearchGate*. DOI: [10.1109/DSAA53316.2021.9564119](#) Accessed: Nov. 16, 2025.
- [93] G. Petri and A. Leitão, “On The Topological Expressive Power of Neural Networks”,
- [94] G. Peyre, “Course notes on Computational Optimal Transport”,
- [95] *Quantitative performance assessment of CNN units via topological entropy calculation (Zhao and Zhang, 2022) - Google Search*, <https://www.google.com/search?client=firefox-b-d&q=Quantitative+performance+assessment+of+CNN+units+via+topological+entropy+calculation+%28Zhao+and+Zhang%2C+2022%29>. Accessed: Nov. 16, 2025.
- [96] *TopoAct: Visually exploring the shape of activations in deep learning (Rathore et al., 2021) - Google Search*, <https://www.google.com/search?client=firefox-b-d&q=TopoAct%3A+Visually+exploring+the+shape+of+activations+in+deep+learning+%28Rathore+et+al.%2C+2021%29>. Accessed: Nov. 16, 2025.
- [97] *Topological data analysis of decision boundaries with application to model selection (Ramamurthy et al., 2019) - Google Search*, <https://www.google.com/search?client=firefox-b-d&q=Topological+data+analysis+of+decision+boundaries+with+application+to+model+selection+%28Ramamurthy+et+al.%2C+2019%29>. Accessed: Nov. 16, 2025.
- [98] *Topological Dynamics of Functional Neural Network Graphs During...* <https://openreview.net/forum?id=4YThDiz3v5>. Accessed: Nov. 16, 2025.
- [99] Y. Zhou, Y. Zhou, J. Ding, and B. Wang, “Visualizing and Analyzing the Topology of Neuron Activations in Deep Adversarial Training”,